The Embodiment Abstraction: A Specification for the Coordination Layer between Vision-Language-Action Models and Robotic Embodiments

Yoichiro Hara

Vox Technologies, Inc.

Comments to: yo@vox.delivery

Project page: `https://github.com/robotfoundationlayer`

RFL Specification v0.1 — 2026-05-30

Contents

Abstract

The physical-AI ecosystem of 2026 faces a coordination problem. Approximately nine credible Vision-Language-Action (VLA) foundation models and approximately thirty distinguishable robotic embodiments yield a combinatorial integration burden plausibly large enough to constrain industrial deployment (no published benchmark currently exists; the paper commits to one in v0.2). Adjacent efforts — ROS 2, Open X-Embodiment, LeRobot, the single-model VLAs themselves, and NVIDIA's Isaac and GR00T stack — each address fragments of the problem but none occupies the specific structural position the situation appears to demand: a cross-vendor, semantically-typed, production-grade, neutrally-governed abstraction layer between intelligence and embodiment. We propose the Robot Foundation Layer (RFL), a three-layer specification consisting of (i) a Skill ISA of approximately fifty primitives with a compositional algebra, (ii) a canonical embodiment-agnostic action representation with a deterministic retargeting interface contract, and (iii) a Driver Interface compatible with ROS 2, all governed by five first principles: embodiment-agnostic, compositional, verifiable, provider-neutral, and forward-compatible. The retargeting contract is the load-bearing technical commitment of the specification; this paper defines the contract and its five binding invariants but leaves the implementing algorithm to the v1.0 reference implementation. We sketch an Appendix B argument that bilateral retargeting quality is plausibly supermodular under standard assumptions, suggesting a structural compounding property whose empirical magnitude is left to deployment. The specification draft, reference-implementation skeleton, and conformance test suite scaffold are released under Apache License 2.0 at the project page; comments and structural critique are solicited through the v0.1 review period.

Keywords

physical AI, vision-language-action models, robotic manipulation, foundation models, abstraction layers, two-sided markets, network effects, standards, ARM ISA, open governance

---

1 § 1. The Coordination Problem

The capability gap between what AI models can describe and what robotic systems can perform has, in the past three years, inverted. As recently as 2023, the binding constraint on physical AI was the model — robots could execute well-defined motions but could not generalize across tasks because the underlying intelligence was too brittle. By the second half of 2025 this constraint had moved. Vision-Language-Action (VLA) foundation models — $\pi 0$ from Physical Intelligence, Gemini Robotics from Google DeepMind, Helix from Figure, GR00T from NVIDIA, OpenVLA from Stanford / UC Berkeley / Toyota Research Institute, and a growing cohort of open-source successors — now generalize across object categories, manipulation primitives, and natural-language instructions in ways that, a decade ago, would have been classified as general AI in the manipulation domain.

The binding constraint has moved to the boundary between these models and the physical bodies they are supposed to inhabit. Every VLA must be re-integrated, by hand, against every embodiment it is to control. Every embodiment vendor must repeat this integration with every VLA it wishes to support. The combinatorial cost has, in 2026, become the dominant drag on the field. This section frames the problem structurally, argues that the present window is the inflection at which the industry must coordinate on a shared abstraction or accept a generation of vertically integrated lock-in, and sets up the remainder of the paper.

1.1 The N $\times$ M Explosion

Two enumerations frame the problem.

On the **model side**, approximately nine VLA foundation model lines are credible candidates for industrial deployment in 2026 — credible meaning they have published checkpoints or commercial APIs, demonstrated multi-task generalization in third-party evaluations, and twelve-month forward maintenance commitments from an organization with public visibility. The set, in alphabetical order: Figure's Helix; Google DeepMind's Gemini Robotics; Hugging Face's SmolVLA and VLA-JEPA lines; NVIDIA's Isaac GR00T family; Physical Intelligence's $\pi 0$ family; Stanford / UC Berkeley / Toyota Research Institute's OpenVLA; Tsinghua's RDT-1B; and the Octo family from UC Berkeley. The count excludes academic checkpoints without industrial maintenance commitments and proprietary models known to be in development at major laboratories but not publicly disclosed.

On the **embodiment side**, the count is larger and growing faster. Restricting to manipulation-capable end effectors and arms with commercial or near-commercial availability as of mid-2026, the set includes humanoid platforms from Tesla, Figure, 1X, Sanctuary AI, Apptronik, Agility Robotics, Boston Dynamics, Unitree, and a cohort of Chinese entrants; multi-finger hands from Inspire Robotics, Wonik (Allegro Hand), CMU (LEAP Hand, open source), Shadow Robot Company, Schunk, Bridgestone (TETOTE pneumatic series), and roughly twenty research-grade hands maintained by academic labs; and conventional collaborative arms from Universal Robots, Franka Emika, Kuka, ABB, UFactory, Doosan, and Yaskawa. Conservatively de-duplicated, this yields approximately **thirty distinguishable embodiments** in active 2026 deployment, with credible projections of fifty by end-of-2027 as the humanoid cohort matures.

The combinatorial implication is direct. Nine VLAs and thirty embodiments generate 270 candidate integration pairs. The realistically valuable subset is smaller — not every pair is commercially meaningful — but still exceeds one hundred pairs today and trends toward two hundred as both sides grow. **No actor in the field is equipped to maintain anything approaching this number of integrations.**

The current response, observable in 2026, is vertical lock-in: each VLA developer prioritizes a small number of embodiments, and each embodiment developer prioritizes a small number of VLAs. The market fragments into eight or ten vertically integrated stacks, each of which captures a fraction of the latent value because no stack achieves the cross-pollination that has driven every previous

wave of platform progress in computing.

1.2 The Integration Cost

A single non-trivial integration — bringing a VLA from supporting one embodiment to supporting an additional one at production-grade reliability — consumes between **two and six engineer-months** of senior robotics-AI talent. The lower bound applies when the new embodiment is structurally similar to one the VLA already supports. The upper bound applies when the embodiment introduces novel actuation modality (pneumatic versus tendon-driven), novel proprioceptive sensors (visuotactile versus force-only), or novel kinematic structure (a six-finger hand instead of a four-finger one).

At prevailing senior robotics-AI engineering wages in the dominant US labor market, this translates to a per-pair direct labor cost ranging from the **low tens of thousands of dollars at the lower bound to the low hundreds of thousands at the upper bound**, with similar ranges in Europe and Asia after currency adjustment. To this must be added three indirect costs:

1. **Hardware procurement and access** — acquiring and instrumenting at least one unit of the target embodiment for the duration of the integration window.
2. **Data collection and labeling** — teleoperation rigs, expert demonstrators, and curation pipelines to produce embodiment-specific demonstration data.
3. **Certification and safety qualification** — ISO 10218 / 13482 assessment when commercial deployment is the target.

The total per-pair cost is **substantial**, and the precise figure varies widely with task complexity, deployment target, and embodiment novelty. No public benchmark exists; the figures here are author estimates intended to convey scale rather than measurement.

1.3 What This Cost Suppresses

The aggregate paid integration cost across the industry is large enough to be macroscopic, but **the more consequential cost is what does not get attempted**. A research lab evaluating whether to test its new VLA on a structurally novel hand must trade six engineer-months against the project's other commitments; in most cases, the test does not happen. A startup considering a new robot form factor must accept that VLA support will lag the hardware by twelve to eighteen months. A potential industrial customer evaluating a robotic solution must price in not only the hardware and software they buy but the integration risk of pairing them.

This **capability suppression** is the largest component of the fragmentation cost and the hardest to measure. It does not appear in any firm's books because it consists of attempts not made. A defensible proxy is the gap between observed and counterfactual market growth in segments where integration cost is binding; this gap is directionally large but resists precise measurement, and we offer no point estimate. What matters for the argument that follows is the qualitative claim, which we take to be uncontroversial: **the opportunity cost of integrations not attempted exceeds, by an order of magnitude or more, the direct cost of integrations made**.

1.4 Why 2026 Is the Inflection Point

The combinatorial pressures described above have existed in some form since the field's inception. What makes 2026 distinctive — and what makes coordination on a shared abstraction layer urgent in this specific window — is the simultaneous occurrence of three structural inflections.

Inflection one: VLA models cross the threshold of cross-embodiment generality. Through 2024, the dominant assumption was that a model trained on one embodiment would transfer poorly to another. The Open X-Embodiment results published by Google and thirty-three partner institutions in 2023, $\pi 0$'s multi-embodiment training in 2024, and Gemini Robotics's transfer results in 2025 falsified this assumption. By 2026 the question is no longer whether a single VLA can support many embodiments but rather how many embodiments will be available to support. The model side is hungrier than the embodiment side can feed.

Inflection two: embodiments diversify faster than they consolidate. The humanoid wave has not produced a single dominant form factor. Tesla Optimus's five-finger tendon design, Figure 03's twenty-four DOF hand, Sanctuary's hydraulic actuation, Bridgestone TETOTE's six-finger pneumatic structure, and the emerging Chinese cohort all represent distinct kinematic and actuation choices, each with a credible industrial sponsor. The expected consolidation toward a single anthropomorphic form has not occurred and shows no sign of occurring on a timeline shorter than a decade. The embodiment side is proliferating, not concentrating, at exactly the moment the model side is demanding it.

Inflection three: capital is committed at unprecedented scale, but coordination capital is absent. Through 2024 and 2025 the physical-AI sector raised disclosed venture financing at a scale measured in tens of billions of dollars, with humanoid robotics accounting for a substantial fraction. This capital has flowed almost entirely into vertical integration — VLA model training, specific robot designs, proprietary data collection — and not into the horizontal coordination layer that would multiply the value of the vertical investments. The field has solved the funding problem for individual stacks and produced a coordination problem in its place.

These three inflections compound. A field with stagnant model generality, consolidating embodiments, or limited capital would not need an abstraction layer urgently — bilateral integration would scale acceptably. A field with all three growing simultaneously, as in 2026, requires a layer that arbitrates the combinatorial growth or it will fragment into stacks that capture, in aggregate, a fraction of the available value.

The historical analog is instructive. The personal-computer industry of the early 1980s faced a structurally similar moment: software was becoming general enough to demand cross-hardware portability, hardware was diversifying rather than consolidating, and capital was abundant. The coordination layer that emerged — the IBM PC architecture, eventually inherited by the x86 ISA and the BIOS conventions that became the de facto standard — captured immense value precisely because it appeared in the brief window when the alternative (per-vendor verticalization) was just barely tolerable. A coordination layer proposed five years earlier would have lacked the demand pull; five years later would have arrived after the lock-in had ossified. The window was eighteen

months, perhaps two years, and the right answer in that window shaped the industry for the next four decades.

A note on what "the window" means for this paper. The 18-to-24-month window is the window during which an *adoption-attracting v0.1 draft* must appear and be visibly anchored in implementer roadmaps — not the window in which a polished v1.0 must ship. The PC analog is again instructive: the IBM PC was announced in August 1981, and the de facto compatibility regime that made the standard binding emerged through 1982–1983 as the first wave of clone manufacturers and software vendors made roadmap commitments against the still-evolving platform. The corresponding RFL milestone is the v0.1 review release (§ 6.1, targeted 2027 Q1), not the v1.0 stabilization release (2027 Q4). v0.1 is the artifact whose adoption decision the window forces; v1.0 is the stabilization that follows after the first wave of implementer commitments. By this reading the relevant question is whether v0.1 can land before the vertical-integration commitments of Figure-Helix, Tesla-Optimus, NVIDIA Isaac-GR00T, and the humanoid cohort have ossified into infrastructure that cannot be re-coordinated against. The honest answer is: marginally, and only if v0.1 is taken seriously by at least three credible cross-vendor implementers within twelve months of release. v1.0 arriving in 2027 Q4 does not by itself "make it" — what makes it is the v0.1-era adoption posture. The roadmap timeline of § 6.1 and the window claim of this section are therefore reconciled by reading "the window" as the v0.1 adoption window, not the v1.0 stabilization horizon.

1.5 The Question Posed

The diagnosis is precise. The physical-AI industry of 2026 is converging on a coordination failure: model generality is high, embodiment diversity is high, capital is high, and the integration interface between models and bodies is bespoke and bilateral. Vertical integration has emerged as the rational firm-level response to the absence of a coordination layer, and the resulting balkanization is locking in the precise structure that long-run progress in physical AI cannot afford.

The remainder of this paper argues that the appropriate response is the construction of a neutral, semantically-typed abstraction layer — the Robot Foundation Layer (RFL) — that occupies the same structural position in physical AI that ARM occupies in silicon, that USB occupies in peripheral interconnects, and that POSIX occupies in operating systems. § 2 reviews why existing efforts (LeRobot, Open X-Embodiment, ROS 2, NVIDIA Isaac, and others) have each failed to occupy this position despite addressing adjacent concerns. § 3 develops the first principles from which the RFL design follows. § 4 presents the specification at version 0.1, including the interface contract for the deterministic retargeting algorithm that anchors the layer. § 5 places RFL in comparative context against ARM, CUDA, USB, ROS 2, and adjacent efforts. § 6 outlines the implementation roadmap. § 7 concludes.

The window is open. It will not remain so indefinitely.

---

2 § 2. Why Existing Approaches Fail to Occupy the Coordination Layer

The coordination crisis described in § 1 is not invisible to the field. Substantial engineering effort over the past five years has gone into systems that address adjacent fragments of the problem, and several have accumulated real momentum. Yet none has converged on the specific structural position that the crisis demands: a cross-vendor, semantically-typed, production-grade abstraction layer governed neutrally between Vision-Language-Action (VLA) models and robotic embodiments. This section examines the five most credible candidates — ROS 2, Open X-Embodiment, LeRobot, the single-model VLAs themselves, and NVIDIA's Isaac/GR00T stack — and argues that each is structurally precluded from occupying the position even on its natural roadmap. The argument is not that these projects are wrong or unimportant; they are correct and important within the problems they were built to solve. The argument is that none of those problems is the same as RFL's problem.

2.1 ROS 2: A Communication Layer Without Semantics

The Robot Operating System, in its second-generation form (ROS 2, built on the Data Distribution Service standard), is the closest thing the robotics field has to a universally adopted convention. Its action-server abstraction, its message-type system, and its publish-subscribe transport are deployed in thousands of research and commercial robots. If any existing system could expand upward into the coordination role RFL would occupy, ROS 2 is the prima facie candidate.

It cannot. ROS 2's design center is the low-level transport of typed messages between processes — "send a `geometry_msgs/Twist` to topic `/cmd_vel`" — and its action-server abstraction defines the *shape* of long-running operations (goal, feedback, result) but not their *meaning*. There is no canonical ROS 2 definition of what it means for a robot to "grasp a cable end and insert it into a connector"; there is only a convention for how such an action's interface might be structured if defined. The semantic gap between "I can transport bytes reliably" and "I can convey the meaning of *grasp* across two different hands so that a single high-level model can drive both" is precisely the gap RFL must close.

Expanding ROS 2 to close this gap is not a roadmap question; it is an identity question. The current ROS 2 contract — backward-compatible, transport-focused, runtime-agnostic — would have to be broken to introduce action semantics, and breaking it would alienate the deployed base on which its credibility rests. The right move for ROS 2 is to remain the transport layer beneath RFL, not to attempt to subsume it. § 4 of this paper makes the integration with ROS 2 explicit: RFL's Driver Interface is designed to expose itself as a set of ROS 2 action servers, so that the existing transport investment is preserved.

2.2 Open X-Embodiment: A Dataset Without a Standard, a Stewardship Without Neutrality

Open X-Embodiment, the Google-led initiative joined by thirty-three partner institutions in 2023, has done more than any other single effort to establish the empirical foundation for cross-

embodiment generalization. The pooled dataset — over a million teleoperated trajectories spanning twenty-two embodiments — supplied the training corpus on which RT-2, $\pi 0$, and subsequent VLAs demonstrated their cross-platform transfer results. The intellectual contribution is substantial.

The structural contribution, however, stops at the dataset boundary. Open X-Embodiment defines a *data format* (largely the RLDS format from Google's prior data work) but not a runtime standard. It tells researchers how to label demonstration trajectories from a new embodiment; it does not tell a production deployment how to route a high-level instruction from a VLA to that embodiment at runtime. A VLA trained on Open X-Embodiment data still requires bespoke integration to control a new embodiment in deployment — the dataset standard does not bridge to a control standard. The same boundary applies to demonstration-capture interfaces such as the Universal Manipulation Interface (UMI), which standardize how manipulation demonstrations are collected in the wild, including the hand-held gripper hardware and the policy-learning pipeline, but not how a model's intent is retargeted to and verified across heterogeneous embodiments at runtime. These are complementary upstream infrastructure for *training* the planner, not the cross-vendor runtime contract RFL occupies.

The second blocker is stewardship. Open X-Embodiment is housed at Google, and Google maintains its own VLA model line (Gemini Robotics) that competes commercially with other VLA providers. This does not impugn the integrity of the academic collaboration, but it does foreclose the trajectory by which Open X-Embodiment could evolve into the neutral coordination layer. The OEM that adopts an abstraction standard housed at a hyperscaler with its own VLA product is making a different commercial bet than the OEM that adopts a standard housed at a neutral foundation. Cisco and 3Com were never going to adopt a networking standard owned by IBM; the same dynamic applies here.

2.3 LeRobot: Strong Momentum, Wrong Center of Gravity

LeRobot, the open-source robotics framework that Hugging Face introduced in 2024 and has aggressively scaled through 2025–2026, is the candidate most often cited in conversations about coordination-layer convergence. It has the right cultural disposition (open source, community-driven), the right developer-experience instincts inherited from the Hugging Face transformers ecosystem, and the most rapidly accumulating contributor base of any project in the field. If RFL is to encounter a serious competitor for the structural position it seeks, the realistic candidate is LeRobot.

The reason LeRobot does not yet — and structurally may not — occupy the RFL position lies in two divergent design pressures. First, LeRobot's center of gravity is the ML-researcher workflow: train a policy, evaluate it on a benchmark, share the checkpoint. This is the right design center for accelerating research, and LeRobot has done it superbly. It is not the right design center for production deployment in industrial environments, which require real-time safety guarantees, certifiability against ISO 10218/13482, deterministic latency, and integration with existing PLC and field-bus infrastructure. The features that make LeRobot a great research framework — Python-

first orientation, flexibility, rapid iteration — work against the deterministic, statically analyzable, safety-certifiable substrate that industrial deployment demands.

Second, Hugging Face's commercial trajectory tends toward vertical platform consolidation rather than standards stewardship. The Hugging Face Hub is itself a platform with proprietary monetization (Spaces, Inference Endpoints, Enterprise tier), and the natural commercial path for LeRobot is to become the gateway to Hugging Face's robotics services. This is a legitimate business model, but it is structurally incompatible with the neutral-foundation governance that the coordination layer requires. An OEM that builds on LeRobot is building on infrastructure whose operator competes commercially with adjacent layers — exactly the trap that ARM avoided by separating itself from Acorn.

A possible counter is that Hugging Face could spin LeRobot out into a neutral foundation analogous to the Linux Foundation. We treat this as the most credible scenario in which the RFL position is contested rather than empty, and § 9's call to action explicitly invites collaboration with LeRobot stewards under such a structure. As of mid-2026 no such restructuring has been signaled.

2.4 Single-Model VLAs: Applications That Cannot Be Platforms

A more provocative claim than any of the above is that the leading VLA models themselves — $\pi 0$ from Physical Intelligence, OpenVLA from Stanford, GR00T N1 from NVIDIA — cannot become the coordination layer regardless of how technically capable they grow. The reasoning is structural, not technical, and it applies even to the most generous interpretation of any of these models' future capability.

A VLA is an application: an artifact that takes perception and instruction in and produces actions out. It competes, in the market, with other applications that do the same thing. Two VLAs competing for the same customer's deployment cannot simultaneously serve as the neutral substrate on which that customer's robot is built. Physical Intelligence's decision to open-source the $\pi 0$ weights does not change this; the weights are now free, but the integration interface — what counts as a valid $\pi 0$ input, what semantics its action outputs carry, how it expects embodiment metadata — remains controlled by Physical Intelligence and follows Physical Intelligence's commercial roadmap. An OEM that builds its embodiment-side software against $\pi 0$'s specific interface is taking a bet on Physical Intelligence, not on a standard.

The dynamic is identical to the early 1990s in operating systems: Sun Microsystems' Solaris was excellent, IBM's AIX was excellent, HP-UX was excellent, but none of them could serve as the substrate for a multi-vendor application ecosystem because each was a commercial product of a vendor that competed for the same customers. The substrate that did emerge — POSIX as a specification, eventually Linux as an implementation — was created precisely because no incumbent could occupy the structural position. The physical-AI field is at the same juncture with respect to VLAs.

The implication is not that $\pi 0$, OpenVLA, and GR00T are unimportant; they are the applications that will run on top of RFL. The implication is that none of them, on its present trajectory, can

become RFL.

2.5 NVIDIA Isaac and GR00T: The CUDA-Adjacent Trap

NVIDIA's Isaac platform, paired with the GR00T N1 humanoid foundation model and the Cosmos world model, is the most technically capable adjacent stack on offer in 2026. It bundles simulation (Isaac Sim, Isaac Lab), perception, motion planning, foundation models, and deployment tooling into the most integrated end-to-end physical-AI offering currently available. By raw capability it is, plausibly, the strongest existing candidate.

By neutrality it is the weakest. Isaac's value proposition is inseparable from CUDA: the simulation runs on NVIDIA GPUs, the foundation models train on NVIDIA hardware, the inference deploys on NVIDIA edge devices (Jetson, Thor). This is not a deficiency in the engineering; it is the engineering. NVIDIA built Isaac to sell NVIDIA silicon, and the architecture reflects that goal at every layer. The result is that any embodiment vendor or VLA provider that builds against Isaac is, structurally, accepting a CUDA dependency in perpetuity.

A coordination layer cannot be CUDA-dependent because half the relevant ecosystem is structurally incentivized to escape CUDA: Apple's Neural Engine, Google's TPU, Tesla's Dojo, Qualcomm's Hexagon, and an emerging cohort of ARM-based and RISC-V-based edge accelerators all need a path to physical-AI deployment that does not route through NVIDIA's tooling. Even within NVIDIA's customer base, the AI-chip market's recent volatility has made customers cautious about deepening any single-vendor dependency. An abstraction layer whose adoption signals long-term lock-in to NVIDIA is, for half its potential adopters, worse than no abstraction layer at all.

The case is parallel to OpenGL versus Direct3D in the late 1990s: Direct3D was technically superior in many releases, but its Microsoft dependency drove the cross-platform graphics market to consolidate around the neutral alternative. RFL's relationship to Isaac is the same: cooperative where possible (RFL implementations should run on Isaac Sim as one of several supported simulators), structurally distinct in governance.

2.6 The White Space, Mapped

Set the five existing efforts against the coordination layer's requirements and the gap becomes geometrically visible.

Requirement	ROS 2	Open X-E	LeRobot	Single-model VLAs	NVIDIA Isaac	**RFL**
Cross-vendor VLA support	n/a	n/a	partial	by definition no	partial	✓
Cross-vendor embodiment support	partial	partial	partial	partial	partial	✓

Requirement	ROS 2	Open X-E	LeRobot	Single-model VLAs	NVIDIA Isaac	**RFL**
Semantically-typed actions (not just messages)	×	×	partial	proprietary	proprietary	✓
Production-grade (real-time, certifiable)	✓	×	weak	varies	✓	✓
Neutral governance	✓	weak (Google)	weak (HF)	by definition no	× (NVIDIA)	✓
Standalone runtime value (not just data/training)	✓	×	✓	✓	✓	✓

Each existing effort satisfies a non-trivial subset of the requirements. None satisfies the full set. The conjunction of *semantically typed*, *cross-vendor on both sides*, *production grade*, and *neutrally governed* defines an empty point in the existing landscape. That point is the position RFL is designed to occupy.

A subtler observation, visible only when the five efforts are considered together, is that the field's coordination has been *over-attempted at the wrong layers* rather than *under-attempted in the aggregate*. There is no shortage of will to coordinate; there is a shortage of will to coordinate at the specific layer where coordination would have the largest network-effect leverage. ROS 2 coordinated transport; Open X-Embodiment coordinated training data; NVIDIA coordinated tooling; LeRobot coordinated research workflow. Each of these coordinations is real and useful. None of them is a coordination of *semantically meaningful action* between *competing model providers* and *competing embodiment providers*. That coordination is what RFL proposes.

2.7 The Implication for Design

Two design implications follow directly from the diagnostic above and govern the choices in § 3 and § 4.

First, **RFL must be additive to existing infrastructure rather than replacement**. ROS 2 is the right transport; Isaac Sim is a fine simulator; Open X-Embodiment data is excellent training material; LeRobot's developer experience is worth borrowing where compatible. The argument of this paper is not that the existing stack is wrong, but that it is incomplete, and that completing it requires a single new piece at a specific layer. The RFL specification is designed to compose with each of the above rather than compete with them.

Second, **the governance question is as load-bearing as the technical question**. The reason every existing effort fails to occupy the coordination position is, in three of five cases, governance rather than technology. A technically perfect specification housed under the wrong steward will not be adopted; a technically average specification housed under the right steward will be. The companion position paper develops the foundation governance structure in detail, but the design pressure is established here: the steward must be a neutral foundation, the spec must be permissively licensed in perpetuity, and the founding members must be drawn from genuinely competitive corners of both sides of the market. The governance structure itself is out of scope for this specification.

The remainder of the paper takes these design pressures as constraints and develops the specification, network-effect architecture, and governance structure that satisfy them.

---

3 § 3. RFL: First Principles

A specification proposed without first principles is a specification that drifts under commercial pressure. Every layer that has achieved durable cross-vendor adoption — POSIX, the C ABI, the ARM ISA, USB, HTTP — was anchored by a small set of design commitments that were stated before the technical details and that constrained the technical details. RFL adopts five such commitments. They are derived from the constraints established in §§ 1–2 (the combinatorial crisis on one side; the structural failures of adjacent efforts on the other), and every choice in § 4's specification can be traced back to them.

This section states the five principles, draws the structural isomorphism with the ARM ISA on which the RFL design pattern is modeled, and identifies the dimensions on which RFL must deliberately diverge from ARM because the physical-AI substrate differs from silicon in ways that compatibility intuition does not capture.

3.1 The Methodological Commitment

Before stating the principles, a methodological commitment: principles, not features. A design principle is a constraint that survives commercial pressure because adherents to the standard refuse to violate it even when individual short-term gains would be available from doing so. A feature is a capability that exists because someone wanted it. Principles can be evaluated against future situations the original drafters did not anticipate; features cannot. The history of every coordination-layer success in computing shows the same pattern: a small set of principles, vigorously defended, outperforms a large set of features, opportunistically accumulated.

The five principles below are the smallest set that, in our analysis, jointly forecloses each known failure mode of past abstraction-layer attempts. We have considered and rejected larger principle sets (the early POSIX standardization process accumulated more than a dozen guiding tenets and slowed correspondingly) and smaller principle sets (a three-principle articulation we initially fa-

vored failed to constrain the governance dimension and would have permitted vendor capture). Five is, on our analysis, the parsimonious count.

3.2 The Five Principles

3.2.1 Principle 1: Embodiment-Agnostic

The specification must avoid encoding the design choices of any particular embodiment. Action semantics must be expressible across tendon-driven, pneumatic, hydraulic, and as-yet-unknown actuation modalities; across four-finger, five-finger, six-finger, and parallel-jaw end effectors; across anthropomorphic and non-anthropomorphic kinematic structures.

This is the foundational constraint and the easiest one to violate. Almost every existing robotics convention is, on examination, quietly encoded with the assumptions of a dominant embodiment class. The Feix grasp taxonomy assumes the human hand; ROS 2's `geometry_msgs` family assumes Cartesian-end-effector control; the dominant teleoperation interfaces assume two-handed bimanual operation. RFL must not bake in any such assumption.

The operational test: a draft of the specification must be reviewable by someone whose only experience of robotics is a non-anthropomorphic platform — say, a pneumatic six-finger hand or a soft-bodied actuator — and they must find the semantics expressible in their world without translation through anthropomorphic intuition. If they cannot, the spec has smuggled in an embodiment assumption and must be rewritten.

3.2.2 Principle 2: Compositional

Skills are atomic primitives that compose, under a defined algebra, into higher-level behaviors. The specification defines the composition rules, not merely the primitive set.

A skill ISA that is only a list of primitives is a skill *library*, not a skill *language*. The difference matters because the value of an abstraction layer grows multiplicatively, not linearly, with the expressiveness of its composition rules. ARM's enduring success rests on its instruction set being composable into procedures, libraries, and operating systems through a small set of calling conventions and addressing modes; the primitives matter less than the composition rules built atop them.

RFL's Skill ISA defines, in addition to the primitive operations, the rules by which sequential, conditional, and concurrent composition is expressed, the way pre- and post-conditions of composed skills are derived from their components, and the algebra by which the safety envelope of a composition is inherited from its components. § 4.4's worked example exercises these composition rules in a single multi-step manipulation; § 4.1 enumerates the primitive set.

3.2.3 Principle 3: Verifiable

Every skill carries pre-conditions, post-conditions, and a safety envelope expressible in a machine-checkable form. Composition rules preserve verifiability.

The verification principle distinguishes RFL from an API. An API dispatches function calls; a verifiable abstraction layer additionally promises that, if the pre-conditions hold and the skill executes to its post-conditions, certain safety-relevant invariants will hold throughout the execution. This promise is what makes the Certification reinforcement loop (§ 5.4 L4) and the Insurance reinforcement loop (§ 5.4 L8) possible. Without verifiability, RFL cannot interoperate with the regulatory and actuarial institutions that physical-world deployment ultimately requires.

The verification principle imposes design discipline on the primitive set. A primitive whose safety envelope cannot be expressed compactly is rejected, regardless of how useful the primitive would otherwise be. This is a real cost; some genuinely useful operations (notably those involving deformable-object manipulation) will lie at the edge of expressibility and require iterative spec refinement. We accept the cost because the alternative — admitting unverifiable primitives — corrupts the certifiability of every composition that includes them.

3.2.4 Principle 4: Provider-Neutral

No vendor's preferred way of doing things has special status in the specification. The specification's evolution is governed by procedures that treat implementers and late entrants identically once admitted.

Provider neutrality is governance expressed at the level of the spec. It is the technical instantiation of the foundation governance principles that the project's stewardship layer is intended to embody (the governance design itself lives in the companion position paper, not in this specification). The principle has two operational tests: first, an outside reviewer should be unable to identify which implementers proposed any given primitive by inspecting the spec; second, the spec's evolution procedure (voting, comment periods, breaking-change moratoria) should produce the same outcome regardless of which member proposes the change.

This principle is what foreclosed the otherwise tempting design choice of letting the most technically capable implementers contribute a head-start spec and have other members ratify it. The technical benefits of such an approach (faster spec maturity, fewer compromises) are real, but the perception of vendor capture — and the actual erosion of cross-vendor adoption that perception would produce — is fatal. The compromise we adopt instead is that the v0.1 reference specification is drafted by the RFL Foundation's neutral spec-editor staff, with proposals from any implementers receiving equal procedural treatment.

3.2.5 Principle 5: Forward-Compatible

The specification is designed to extend without breaking. Future VLA capabilities and future embodiment classes must be expressible in extensions; the v1.x line must never deprecate v1.0 functionality.

The forward-compatibility principle is the discipline that distinguishes a standard from a moving target. Backward compatibility within a major version is not optional. New primitives, new composition rules, and new metadata fields enter the spec through additive extensions, organized into a versioned extension registry analogous to RISC-V's modular extensions or x86's CPUID feature

bits. A VLA that targets RFL v1.0 in 2027 must continue to work against RFL-compliant embodiments shipped in 2032; an embodiment certified against RFL v1.0 must continue to receive VLA support indefinitely.

This principle imposes a non-trivial intellectual cost on the v0.1 design: the drafters must anticipate which extensions are foreseeable and structure the core spec so that those extensions can be added without disturbing existing semantics. We have used the design heuristic of *capability discovery* (every RFL-compliant component exposes its capabilities through a structured manifest) and *graceful degradation* (a VLA that requires a capability the embodiment lacks must receive a clear failure mode, not undefined behavior). § 4.3 develops both mechanisms in detail.

3.3 Isomorphism with the ARM ISA

The five principles, taken together, place RFL in structural correspondence with the ARM Instruction Set Architecture. The correspondence is close enough to be generative for design — i.e., the ARM case can be used as a check on whether a proposed RFL design choice has a plausible precedent — but it breaks down at the divergences enumerated in § 3.4, the most consequential of which is the absence of any ARM analog for physical safety. Read accordingly: the table below is a *partial* point-for-point correspondence at the layer where the analogy is informative, not a complete isomorphism. Other coordination-layer precedents (USB for consortium governance, POSIX for portability discipline, OpenSSF for safety-relevant stewardship) offer partial correspondence to the dimensions where ARM does not, and are referenced where appropriate in § 5.

RFL element	ARM ISA element
Skill ISA primitives	Instruction set primitives
Skill composition rules	Calling conventions, ABI
Pre/post-condition contracts	Architectural invariants (e.g., register state preservation)
Driver Interface	Memory model, peripheral I/O conventions
Capability discovery manifest	CPUID-equivalent feature reporting
Extension registry	ARM architecture profiles (A/R/M, plus extensions)
Foundation governance	ARM Holdings as steward
Compliance test suite	ARM architecture validation suite

The correspondence is generative, not accidental. The first principles in § 3.2 were derived in part by asking, for each ARM ISA design choice that proved durable: *what was the structural reason that choice succeeded, and what is the physical-AI analog of that reason?* Embodiment-agnosticism corresponds to ARM's silicon-agnosticism (the ISA does not specify how the chip is fabricated); compositionality corresponds to ARM's procedure-and-library compositionality (the ISA defines how procedures call each other); provider-neutrality corresponds to ARM Holdings' decision not to manufacture

chips itself; forward-compatibility corresponds to ARM's discipline of architectural-version stability across decades.

The historical evidence is strong. ARM's enduring position in silicon emerged precisely from disciplined adherence to a small set of principles that resisted commercial pressure to deviate. Two specific moments are illustrative: ARM's refusal to acquire chip-fabrication capacity in the early 2000s (which would have grown short-term revenue but destroyed the neutrality that anchored vendor adoption); and ARM's decision in the late 2010s to extend the architecture (ARMv8, ARMv9) through additive features rather than breaking changes (which preserved the multi-decade application binary base). RFL's principles 4 and 5 are direct descendants of those decisions.

3.4 Where RFL Must Diverge from ARM

The isomorphism, however, is not complete. The physical-AI substrate differs from silicon along four dimensions that the ARM design did not need to address and that RFL cannot inherit unchanged.

Safety as architectural concern. ARM's instructions do not physically harm. A buggy ARM program can hang the machine, corrupt data, or leak information, but it cannot crush a human hand. RFL's primitives can. The verifiability principle (Principle 3) and the safety envelope mechanism it requires have no ARM analog and must be designed from first principles for physical action. § 4.2's translation layer and § 4.4's worked example show the safety envelope's structure.

Two-sided market urgency. ARM emerged into a market where chip vendors and OS providers were not racing each other to vertical integration. The silicon ecosystem had time to consolidate around the ISA. RFL faces a market in which vertical integration (Figure–Helix, Tesla–Optimus, NVIDIA–GR00T) is already underway and accelerating; the window for a horizontal coordination layer is narrower than ARM's was, and the founding-member commitments must be secured before vertical lock-in ossifies. This urgency shapes the cold-start strategy in § 5.3 and the consortium recruitment plan in § 9.

Data gravity as a moat input. Silicon does not generate data that improves silicon. Physical AI does: every executed action becomes training data for the next generation of translation models. RFL's design must therefore include the data-flow architecture that captures and shares this learning across the ecosystem without violating the privacy or commercial interests of the participating implementers. § 4.5's conformance test suite and the project's data-sharing governance (out of scope for this specification) both descend from this concern.

Multi-jurisdictional regulatory exposure. ARM's most-cited state-level pressure was export controls on advanced cores, but the broader ARM story also includes recurring antitrust scrutiny of its licensing terms (notably during the failed NVIDIA acquisition attempt in 2020–2022) and ongoing standards-essential-patent (SEP) tensions with licensees over royalty fairness. We do not mean to suggest ARM was regulation-free; we mean that ARM's regulatory exposure clustered around a small number of well-defined surfaces (export, licensing fairness, SEP) that the stewarding organization could address through licensing-policy changes without architectural changes to the

ISA itself. RFL faces a structurally denser thicket whose burden lands on the *technical* specification rather than only on the stewarding entity's policies: ISO 10218 and 13482 on industrial-robot safety, the emerging EU AI Act provisions on autonomous systems, sector-specific medical-device regulation for healthcare embodiments, and a patchwork of national rules on AI accountability. The forward-compatibility principle (Principle 5) acquires a regulatory dimension that the ARM ISA itself never had: the spec must be extensible not only to absorb future capabilities but to absorb future regulatory frameworks without architectural disruption.

Each of these divergences makes RFL harder to design than ARM was. None of them invalidates the structural correspondence. The five principles remain the spine; the divergences are constraints on how the spine is fleshed out.

3.5 How the Principles Constrain the Specification

The principles are not aspirational; they are operational. § 4's specification is constructed under their constraint in the following specific ways.

The Skill ISA primitive set is restricted to operations whose pre/post-conditions and safety envelopes are expressible in the verification language (Principle 3) and whose semantics are stable across at least three structurally different embodiments in the v0.1 reference implementations (Principle 1). The Translation Layer's canonical action space is parameterized along axes that admit any kinematic and actuation modality currently known to be commercially feasible, rather than along axes that are convenient for a specific class of robots (Principle 1). The Driver Interface contract is designed so that any robot whose firmware exposes the contract can host any RFL-compliant VLA, without negotiation specific to that VLA (Principle 4). Every primitive carries a version stamp from its first appearance, and the v1.x evolution policy explicitly forbids retracting a stamped primitive (Principle 5). The composition rules are defined such that the safety envelope of a composed skill is computable from the safety envelopes of its components, enabling certification at composition granularity rather than requiring per-composition reverification (Principles 2 and 3).

These constraints make § 4 longer and more conservative than a feature-driven specification would be. They are the deliberate cost of building a layer that can survive the commercial pressure it will encounter, in the same way that ARM's disciplined architectural conservatism imposed costs in 1985 that paid off across the four decades that followed.

---

4 § 4. The RFL Specification v0.1

This section presents the Robot Foundation Layer specification at version 0.1. It is deliberately narrow in scope: it specifies the abstraction layer between Vision-Language-Action (VLA) intelligence and robotic embodiment, and nothing else. Layers above the spec (task planning, world modeling) and below it (real-time motor control, sensor drivers) are explicitly out of scope, in accordance with the Layer Discipline principle (§ 5.8, derived from Principle 4). The spec defines three layers: a Skill

Layer providing semantically typed action primitives and their composition algebra; a Translation Layer providing a canonical, embodiment-agnostic action representation; and a Driver Interface defining the contract between the Translation Layer and embodiment-specific firmware.

The spec is presented at a level of detail sufficient to fix the architectural commitments and to enable competent implementers to begin work. Full implementation detail — every type signature, every JSON schema, every error code — is deferred to the companion GitHub repository, where the spec lives as a versioned document subject to the project's spec evolution procedure (described in the companion governance document, out of scope for this specification).

4.1 The Skill Layer

The Skill Layer defines fifty base primitives organized into seven categories, plus the compositional algebra that combines them into multi-step manipulations. The primitive count is itself a design commitment: too few primitives forces VLAs to express common manipulations through awkward compositions, while too many primitives encourages embodiment-specific encoding (Principle 1). Fifty is the count at which, in the authors' internal v0.1 design review, every commonly cited industrial manipulation could be expressed in three or fewer composition steps and no primitive applied to only a single embodiment class. **This count and the categorical decomposition have not yet been red-teamed by independent practitioners across structurally distinct embodiment classes (tendon-driven, direct-drive, pneumatic, soft-body); the v0.1 review period is structured to surface omissions and redundancies, and the count is expected to shift modestly before v1.0 freezes.**

The categories and representative primitives:

Category	Count	Representative primitives
Grasping	10	`grasp_power`, `grasp_precision`, `grasp_pinch`, `grasp_lateral`, `grasp_hook`, `grasp_cylindrical`, `release`, `regrasp`, `transfer_grasp`, `adjust_grip`
Translation & placement	8	`place`, `push`, `pull`, `slide`, `drag`, `stack`, `retract`, `advance`
Rotation & orientation	6	`rotate`, `twist`, `screw_in`, `screw_out`, `reorient`, `align`
Insertion & connection	8	`insert`, `seat`, `mate`, `snap`, `latch`, `clip`, `lock`, `plug`
Continuous manipulation	8	`pour`, `wipe`, `fold`, `unfold`, `smooth`, `crease`, `drape`, `wrap`
Inspection & sensing	6	`probe`, `palpate`, `scan`, `measure_force`, `measure_compliance`, `verify_state`

Category	Count	Representative primitives
Multi-effector coordination	4	`bimanual_handoff`, `bimanual_hold_and_act`, `multi_finger_regrasp`, `cooperative_carry`

Each primitive is specified through five fields: a *type signature* (typed inputs and outputs), *pre-conditions* (environmental and state predicates that must hold before invocation), *post-conditions* (predicates that hold upon successful completion), a *safety envelope* (force bounds, motion bounds, and abort triggers that hold throughout execution), and a *capability requirement* (the embodiment capabilities, declared through the Driver Interface, that the primitive requires for execution).

As a concrete example, the `grasp_pinch` primitive is specified as follows:

```
primitive grasp_pinch:
  inputs:
    target:       Object3D            # geometry + pose + material descriptor
    force_budget: Force[N]            # default 5.0 N, max 50.0 N
    grasp_pose:   Pose6D | "auto"     # default "auto" (planner-derived)
  outputs:
    result:       {success, slip, force_exceeded, unreachable}
    final_pose:   Pose6D              # actual grasp pose achieved
    telemetry:    GraspTelemetry      # tactile + force trace
  preconditions:
    - target.is_visible_to(embodiment.perception)
    - target.estimated_mass <= embodiment.capabilities.payload_grasp_pinch
    - embodiment.state.end_effector_free
  postconditions:
    - if result == success: object_held(target, mode=pinch)
    - if result == slip:    embodiment.state.end_effector_free
  safety_envelope:
    - peak_force <= force_budget * 1.2         # 20% transient margin
    - peak_velocity <= embodiment.limits.v_grasp
    - abort_if: target.crush_indicator > 0.8
  capability_requirement:
    - {pinch_grasp: required}
    - {tactile_sensing: preferred, level: "binary_contact" or higher}
```

The specification structure is uniform across all fifty primitives, which is what enables the verifiability principle (§ 3.2 Principle 3): a tool can mechanically check that every primitive in a composition has its pre-conditions established by predecessors and that the composition's safety envelope is the proper composition of the individual envelopes.

Compositional algebra. The Skill Layer defines five composition operators, sufficient to express the manipulation workflows we examined across the v0.1 design review.

Operator	Syntax	Semantics
Sequential	`A ; B`	Execute `A` to success, then execute `B`; if `A` fails, composition fails
Conditional	`A ? B : C`	Execute predicate `A`; on true execute `B`, on false execute `C`
Concurrent	`A ‖ B`	Execute `A` and `B` simultaneously; safety envelope is union; composition succeeds when both succeed
Reactive	`A until trigger`	Execute `A` until `trigger` fires, then halt `A` cleanly
Bounded retry	`try A retries n`	Execute `A`; on failure retry up to `n` times before failing the composition

Composition rules preserve verifiability. The pre-condition of a composition is the pre-condition of its first-executed component; the post-condition is the post-condition of the last component along the executed path; the safety envelope is the union of the components' envelopes for `;` and `?`, and the strict intersection for ‖ (any envelope violation by either concurrent action aborts both). Bounded retry inherits the envelope of `A` unchanged but admits multiple attempts within it.

A subtlety worth noting: the Concurrent operator's strict-intersection safety semantics is deliberately conservative, because two concurrent actions whose envelopes individually permit certain forces may, when executed together, produce force interactions that neither envelope anticipated. The conservative choice forecloses a class of compositions that would, in some embodiments, be safe; we accept the cost as the price of mechanical verifiability.

4.2 The Translation Layer

The Translation Layer converts Skill Layer invocations into a canonical, embodiment-agnostic action representation that the Driver Interface can route to embodiment-specific firmware. It is the spec's most architecturally novel layer, occupying the position that ROS 2 does not (semantic translation, not just message transport) and that single-model VLAs cannot (cross-embodiment routing, not just internal model dispatch).

The canonical action representation is a tuple:

```
CanonicalAction = (
  pose_trajectory:     SE(3) sequence with timestamps,
  force_trajectory:    Force[6] sequence aligned with pose_trajectory,
  tactile_target:      TactileManifold | None,
```

```
  proprioceptive:      JointConfig | None,
  capability_flags:    set of CapabilityFlag,
  confidence:          DistributionOverSuccess,
  safety_envelope:     SafetyEnvelope (inherited from Skill Layer)
)
```

Five aspects of this representation are load-bearing.

First, the pose and force trajectories are expressed in the *task frame*, not in any embodiment-specific frame. The Driver Interface's calibration handshake (§ 4.3) is responsible for mapping task-frame coordinates into the embodiment's native frame at execution time. This indirection is what allows the same canonical action to drive radically different embodiments without retargeting at the VLA layer.

Second, the tactile target is expressed in the TactileManifold abstraction — a topological description of contact patterns rather than a specific tactile-sensor output format. An RFL-compliant embodiment whose tactile sensors produce four-channel binary contact data can describe its observations in TactileManifold terms; an embodiment with high-resolution GelSight-class visuotactile output can describe its richer observations in the same terms with higher resolution. The abstraction is leaky in a controlled way: VLAs can request a minimum manifold resolution through capability flags, and the Translation Layer's capability negotiation will route the action to embodiments that meet the requirement (Principle 5: graceful degradation).

Third, the proprioceptive field is *optional*. Many Skill Layer primitives — particularly grasping and continuous manipulation — do not require joint-level specification, because the Translation Layer can derive a kinematically feasible joint trajectory from the task-frame trajectory and the embodiment descriptor. The field is present for the cases where the VLA has a specific reason to control joint configuration (avoiding obstacles, matching a learned policy's training distribution).

Fourth, the confidence field carries a distribution over outcomes rather than a point estimate. This is the input the Driver Interface needs to set appropriate safety margins: a high-confidence action can execute with aggressive timing, a low-confidence action triggers conservative timing and richer telemetry capture.

Fifth, the safety envelope is inherited unchanged from the Skill Layer composition. The Translation Layer is responsible for ensuring that the canonical action it produces will not, when executed, violate the envelope; if no kinematically feasible trajectory exists within the envelope, the Translation Layer returns a failure to the Skill Layer rather than relaxing the envelope.

Retargeting interface contract. The mapping from a Skill Layer invocation to a canonical action depends on the target embodiment. The Translation Layer is parameterized by an *embodiment descriptor* (loaded from the Driver Interface at session initialization). The v0.1 specification **fixes the interface contract for the retargeting function and defers the algorithmic implementation to v1.0** (2027 Q4 target). The interface is:

```
retarget : (Skill, EmbodimentDescriptor, EnvironmentState)
```

$\rightarrow$ `CanonicalActionSequence` $\cup$ `{`$\perp$`_safety, `$\perp$`_unsupported}`

Any conformant implementation must satisfy five **binding invariants**:

ID	Invariant	What it requires
I1	**Determinism**	`retarget` is a pure function; identical inputs produce byte-identical output across all conformant implementations on all platforms. No reliance on system clock, RNG, or thread scheduling permitted in the produced canonical-action bytes. (See note below on the engineering envelope required to achieve byte-level determinism in the presence of floating-point arithmetic.)
I2	**Embodiment-respect**	Every action in the output is executable by the embodiment described by the input descriptor under its declared kinematic and actuation envelope. The Translation Layer never emits an action the embodiment cannot perform.

ID	Invariant	What it requires
I3	**Composability (scoped to rest-stable boundaries)**	For every pair `(s1, s2)` whose boundary is *rest-stable* (Definition I3.1 below), `retarget(sequence(s1, s2), e, env) = concat(retarget(s1, e, env), retarget(s2, e, env'))` where `env'` is the environment state after `s1` executes. Analogous distributivity over `parallel`, `repeat`, and `branch` constructors, restricted to operands meeting the corresponding rest-stable preconditions. I3 does **not** apply across boundaries that carry residual momentum, ongoing contact forces, or unbroken in-hand grasps; the Skill Layer makes such boundaries explicit through dedicated primitives (`stabilize`, `release`, `settle`) that re-establish rest-stability before composition. See "Note on I3" below for the definition, the rationale for the restriction, and a worked decomposition in § 4.4.
I4	**Failure-mode preservation**	If `retarget` cannot satisfy the skill's safety envelope under the target embodiment, it returns `⊥_safety` explicitly rather than emitting a degraded or partially-safe action sequence. Silent envelope relaxation is non-conformant.
I5	**Extension-namespace isolation**	Extension primitives (`ext.<namespace>.<name>`) flow through `retarget` unchanged when the embodiment descriptor declares support; `⊥_unsupported` is returned otherwise. The core retargeter does not interpret extension semantics.

The conformance suite (§ 4.5) defines two relevant test classes against these invariants:

- **Class 2: Translation Layer reproducibility** — for a fixed `(s, e, env)` triple, observed output across runs and platforms must agree per the regime the implementation declares: **Class 2-strict** requires byte-equality (the trademark-gated regime); **Class 2-loose** requires bounded-difference equivalence under per-skill `(d_skill, ε_skill)` (the non-trademark-gated regime that admits native-precision learned retargeters). See the "Note on I1" and the "v0.1 design decision: Split conformance regime" subsections below. Both subclasses are testable against any candidate implementation, not only the eventual v1.0 reference, and constitute the earliest layer of the spec to which conformance can be evaluated.
- **Class 4: End-to-end execution** — exercised against actual or high-fidelity-simulated embodiments, requires I2 + I4 by construction.

The v1.0 reference implementation will be released under Apache 2.0 and exposed through both a Python binding (for ML integration) and a C binding (for embedded integration). Its source is intended to serve as the canonical-by-construction definition, in the sense that any divergence between the reference implementation and a third-party implementation on a Class 2 test is, by convention, a defect of the third-party implementation rather than of the reference.

Why interface-only at v0.1. Fixing the interface contract while deferring the algorithm is a deliberate sequencing choice. The five invariants above are the load-bearing claims: I1 is what makes conformance testing possible at all; I3 is what makes the compositional algebra of § 4.1 non-vacuous; I2 + I4 are what make the layer safe to deploy in industrial settings. The algorithm satisfying these invariants admits many implementations (inverse-kinematics-first, sampling-based, learned, hybrid); rather than freeze one in v0.1 and inhibit experimentation, the spec freezes the contract and lets v0.1 implementers compete on substrate while remaining conformant. The reference implementation in v1.0 will be one such substrate, not the substrate.

Note on I1 (byte-level determinism in practice). Byte-identical output across platforms in the presence of floating-point arithmetic is achievable but is a non-trivial engineering envelope. The v1.0 reference implementation is expected to satisfy I1 by: (1) using IEEE 754 binary64 throughout, with explicit disabling of fused-multiply-add (FMA) optimizations in any loop whose output enters the canonical-action bytes; (2) serializing all transcendental function calls through a portable software library (rather than platform `libm`, which is non-determinant across vendors); (3) imposing a strict ordering on any iteration over unordered collections; and (4) emitting all numeric fields in the canonical action through a portable text formatter rather than via direct binary serialization. These constraints are well-known from cryptographic-hash and consensus-protocol design and are tractable for a coordination-layer implementation; they are listed here to acknowledge that I1 is a *binding engineering envelope*, not a free property of the language runtime. Conformance Class 2 tests this envelope by running the reference test corpus across at least three platform-toolchain pairs (Linux/x86_64, macOS/arm64, Windows/x86_64) and asserting byte-identical output.

Note on I1 and learned retargeters — an acknowledged tension. The phrase above about "competing on substrate (inverse-kinematics, sampling, learned, hybrid)" must be read in light of I1. Neural-network inference on contemporary GPU stacks is *not* bytewise deterministic across platforms — non-associative floating-point reductions, vendor-specific kernel selection, and CUDA-

stream non-determinism all reliably produce per-platform bit differences. A retargeter whose hot path is a GPU-resident learned model at native floating-point precision therefore *cannot* satisfy I1 as stated, regardless of training. The spec at v0.1 does not pretend to dissolve this tension; it makes the trade-off explicit:

- *Permitted under I1.* Classical retargeters (inverse kinematics, sampling, optimization-based); learned retargeters whose outputs are *rounded into the canonical-action quantization grid* such that residual platform float-difference falls below the rounding threshold; learned retargeters that run on a deterministic CPU-only inference backend (e.g., a well-controlled integer-quantization path); learned post-processors applied to deterministic core output where the post-processor's output is similarly quantized.
- *Not permitted under I1 as stated.* End-to-end GPU-resident learned retargeters operated at native floating-point precision without an explicit quantization-into-canonical-bytes step.

This is a substantive constraint on the substrate over which v0.1 implementers may compete. We adopt it because the alternative — relaxing I1 to "approximately deterministic" — destroys the property's value entirely: a Class 2 conformance test cannot meaningfully compare two implementations whose outputs may legitimately differ by an unbounded amount. Bytewise determinism is the property that makes the spec testable at all, and the implementers it excludes — GPU-resident learned retargeters at native precision — would not pass any other reasonable composability or reproducibility test either. The v0.1 review period explicitly solicits comment on whether this trade-off should be relaxed (e.g., by replacing byte-equality with a bounded numerical-difference predicate in I1) or preserved as stated.

A specific and safety-relevant consequence of the I1 commitment is worth stating plainly. The force-related envelope bounds, most consequentially the minimum holding force that keeps a grasped object from slipping, are derived from a declared friction coefficient and a schematic capacity model, and are never measured at runtime: a runtime measurement would make the retargeting output depend on sensor noise and violate I1. Real slip depends on contact-patch deformation, surface contamination, and load history that a single declared coefficient does not capture. RFL accepts this as the price of determinism: the contract fixes a reproducible, auditable lower bound, and an embodiment that needs a tighter guarantee layers runtime force sensing above the contract rather than inside it.

The deeper structural concern: I1 versus Principle 5 (forward-compatibility). The trade-off above is presented as a substrate choice, but a more pessimistic reading is available and we name it explicitly: if the trajectory of physical-AI research is toward *end-to-end learned retargeters* operated at native GPU precision — and the broader VLA trajectory of 2024–2026 is consistent with that direction — then the choice strict-I1 *forces* (deterministic CPU-only or quantized inference) is not on the dominant path of the field. A spec that locks its only conformance class against the direction of substrate evolution does not survive its own forward-compatibility commitment.

v0.1 design decision: Split conformance regime. Earlier iterations of this section presented three candidate positions (preserve I1 strict; relax I1 to bounded-difference everywhere; split the regime)

as open issues for the review period. On reflection — and in response to external review — leaving this open at v0.1 would push the structural collision onto the first wave of implementers, who would have to bet on which way the spec resolves before they can decide what to build. We commit at v0.1 to the **split regime** (the position labelled "C" in earlier drafts) and define two named conformance classes:

- **Class 2-strict** — byte-equality. The retargeting output must be bit-identical across all conformant implementations on all platforms, exercised across at least three (OS, architecture) pairs. This is the class the RFL™ trademark gates against (§ 6.3 Tier 2/3); it is the class certification-critical deployments (ISO 10218/13482, EU AI Act high-risk, medical-device contexts) are expected to require; it is what makes the spec mechanically verifiable. Implementations satisfying Class 2-strict are typically classical retargeters (IK, sampling, optimization), CPU-only or integer-quantized learned retargeters, or learned post-processors whose output is rounded into the canonical-action quantization grid.
- **Class 2-loose** — bounded-difference equivalence. The retargeting output must agree across conformant implementations to within a per-skill numerical tolerance ε`_skill` measured under a per-skill metric `d_skill` on canonical actions; the tolerance and metric for each of the fifty primitives are specified in a separate `loose-tolerance.toml` published alongside v0.1 and updated each minor version. Implementations satisfying Class 2-loose but not Class 2-strict are admitted to the spec's compositional and discovery machinery but are *not* eligible for the RFL™ trademark and are flagged in the public registry as `Conformance: 2-loose only`. This is the class that admits end-to-end GPU-resident learned retargeters at native precision.

The split is the architectural commitment; the per-skill ε`_skill` values and the metric `d_skill` definitions are open for the v0.1 review period and will stabilize at v1.0. The reasoning behind committing at v0.1 rather than deferring: leaving the regime open imposes a coordination problem on implementers (which class do I target?) that the spec itself can resolve cleanly by naming both classes and the trademark/regulatory boundary between them. The cost of this commitment is that v1.0 will need to publish the per-skill tolerance table; we accept that cost as the price of letting implementers begin work against a stable conformance taxonomy.

Note on I3 (composability — scope, definition, and limits). I3 as originally stated (unrestricted distributivity of `retarget` over `sequence`) is *false* for the retargeting problem in physical robotics. Any naive composition that ignores environment-state carryover, residual momentum, or in-hand object dynamics will produce counter-examples. The scoped form in the table above is what the spec actually commits to, and the scope is load-bearing — readers who confuse the scoped form for the unrestricted form will misjudge the spec's claim.

Definition I3.1 (rest-stable boundary). A boundary between skills $\mathtt{s}_1$ and $\mathtt{s}_2$ in a sequence is *rest-stable* if all four of the following hold at the boundary point (i.e., at the moment $\mathtt{s}_1$ terminates and $\mathtt{s}_2$ is about to begin):

1. *Zero residual velocity.* All embodiment joints and the end-effector are at zero commanded velocity, and any observed velocity is below the embodiment's noise floor (declared in the

Driver Interface).

2. *No active grasp.* No object is held under closed grasp constraints; if an object was held during s_1, it has been released, placed, or transferred before the boundary.
3. *No active contact forces.* Sustained contact between the end-effector and any object is broken; the embodiment is in free space or in resting contact under gravity only.
4. *Environment quiescence.* No object in the workcell is in commanded motion or in mid-trajectory; passive settling under gravity has completed (typical settling time: 200–500 ms after s_1's terminal action).

When s_2's precondition is fully expressible in terms of the discrete world state after s_1 (which objects are where, in what poses, gripped by whom), and the continuous state at the boundary satisfies (1)–(4), the boundary is rest-stable. Under this condition, retargeting s_2 against the post-s_1 environment yields the same byte sequence regardless of whether s_1 was previously retargeted in the same call or computed independently — which is exactly what I3 asserts.

Why the restriction. The unrestricted distributivity claim fails most visibly in three classes of decomposition:

- *Momentum-carrying boundaries.* If s_1 is a fast pick-and-place and s_2 is a follow-on placement at a nearby pose, an optimized retargeter for the joint skill $s_1;s_2$ can exploit the terminal momentum of s_1 to reduce the time and energy of s_2's startup. The independent retargetings of s_1 and s_2 separately cannot, because each must assume rest-to-rest. Byte-equality is therefore unattainable across the boundary, and the composition produces a *different (and superior) plan* than the concatenation of independent plans. The Skill Layer encodes this via the `momentum_chain` primitive in the manipulation category, whose decomposition explicitly opts out of I3.
- *Grasp continuity.* If s_1 ends with an object in grip and s_2 continues to manipulate the same object, retargeting s_2 independently from s_1 would re-plan the grasp from scratch — losing the in-hand pose information that the joint retargeter would preserve. The Skill Layer addresses this through `transfer_continuation` (an explicit handoff primitive that serializes in-hand state into the canonical action's confidence field) and through the `release; reacquire` composition pattern, which restores rest-stability at the cost of one to two seconds of cycle time.
- *Mid-contact boundaries.* If s_1 ends mid-insertion and s_2 continues the insertion, separating the retargetings loses the contact-force history that the unified retargeter would use to set initial impedance. The Skill Layer's `stabilize` and `settle` primitives provide explicit rest-stability points; mid-contact boundaries without them are non-conformant.

The boundary discipline this imposes on Skill Layer composition is intentional. It is the same discipline that distributed systems and compiler optimizers impose for analogous reasons (sequential consistency requires explicit synchronization points; loop fusion requires absence of cross-loop dependencies). § 4.4 below works through one explicit decomposition of the cable-insertion task to make the discipline concrete.

What "scoped to rest-stable" buys the spec. Because rest-stable boundaries are decidable from the canonical action's terminal frame plus the environment quiescence check (both observable at the Driver Interface), Class 2 conformance can test I3 mechanically: for every sequence in the conformance corpus whose boundary is rest-stable, the suite retargets the sequence both ways (jointly and as concat-of-parts) and asserts byte-equality. Sequences whose boundaries are not rest-stable are excluded from the I3 test class but are tested under I2 + I4 only. This is a weaker compositional property than the unrestricted version would have been, but it is the strongest compositional property that admits a mechanical test on the physical-robotics retargeting problem.

4.3 The Driver Interface

The Driver Interface defines the contract between the Translation Layer and embodiment-specific firmware. It has three components: extended kinematic description, capability discovery, and a runtime action protocol.

Extended kinematic description. RFL extends the URDF/MJCF kinematic-description standard with additional fields required for the spec's purposes:

- Tactile sensor placement and modality (location on the kinematic tree, manifold resolution, native data format)
- Actuation modality (electric, pneumatic, hydraulic, tendon-driven) and its associated dynamic properties (bandwidth, hysteresis, backlash)
- Proprioceptive sensor characterization (joint encoder resolution, force-torque sensor placement and bandwidth)
- Safety-relevant kinematic limits (joint velocity ceilings, end-effector workspace bounds, collision-detection model)
- Calibration metadata (when the embodiment was last calibrated, against which reference, with what residual error)

The extension is additive — a standard URDF parser produces a usable but incomplete embodiment descriptor; an RFL-aware parser produces the full descriptor. This composability with existing tooling is itself a Principle 4 commitment.

Capability discovery. Every RFL-compliant embodiment exposes, at session initialization, a manifest declaring which Skill Layer primitives it can execute, with what fidelity, and under what conditions. The manifest is structured:

```
capability_manifest:
  primitives_supported:
    grasp_pinch:
     fidelity:       "production"          # "experimental" | "stable" | "production"
      success_rate:    0.95                    # empirical on conformance test
      latency_p95:     0.8 seconds
      payload_max:     0.5 kg
      object_size_min: 5 mm
```

```
    object_size_max: 80 mm
    tactile_level:   "binary_contact"
  grasp_power:
    fidelity:        "production"
    ...
capabilities_declared:
  {pinch_grasp, power_grasp, lateral_pinch, tactile_sensing,
   force_sensing_3axis, dual_arm: false}
extensions:
  [rfl-tactile-highres-v0.1]                # opt-in extensions
```

The manifest is the basis for capability negotiation between the VLA and the embodiment. A VLA targeting `grasp_pinch` against an embodiment whose manifest declares `fidelity: experimental` knows to set conservative confidence; an embodiment whose manifest does not declare `pinch_grasp` capability is matched against a different primitive (`grasp_lateral` is the canonical fallback) through the Translation Layer's retargeting algorithm.

Runtime action protocol. At runtime, canonical actions are dispatched to the embodiment through a ROS 2 action server interface. This is a deliberate compatibility commitment with the dominant transport standard (§ 2.1): every RFL-compliant embodiment is, by virtue of compliance, exposed as a set of ROS 2 action servers, one per supported primitive. The action server's goal message carries the canonical action; the feedback messages carry execution telemetry at a declared cadence; the result message carries the outcome plus a structured execution trace. The trace structure is itself standardized — it is consumed by the conformance test suite and by certification tooling, and its standardization is what enables third-party audit of RFL-compliant behavior.

The choice of ROS 2 as the wire protocol does not preclude alternative transports (gRPC, MQTT, DDS-other) for embodiments that do not run ROS 2 internally; the spec defines a wire-format-neutral message structure and provides reference bindings for each major transport. ROS 2 is the default because it is the deployed standard, not because it is privileged in the spec.

4.4 Reference Retargeting Recipe — An Existence Proof for One (Skill, Embodiment) Pair

A common objection to interface-only v0.1 specifications is that an unimplemented contract is indistinguishable from an unsatisfiable one. To answer it, we specify here a concrete retargeting recipe for a single (skill, embodiment) pair — `grasp_pinch` on the four-finger tendon-driven Allegro Hand — and walk through how the recipe satisfies each of I1–I5. The recipe is an algorithmic specification, not a complete implementation; the v1.0 reference implementation will instantiate it (and recipes for the other 49 primitives) in Rust with portable transcendentals. The purpose here is to demonstrate that **a five-invariant-satisfying retargeter for at least one non-trivial primitive on at least one real embodiment is constructible**, which is the weakest existence claim the spec is committed to and the one whose absence the reviewer of an interface-only spec is right to challenge.

Setup. The skill `grasp_pinch(target=T_target, force_budget=F_max, approach_axis=a)` instructs the embodiment to acquire a stable pinch grasp on an object whose contact pose is specified in the task frame as `T_target`, applying no more than `F_max` of normal force, with end-effector approach along axis `a`. The Allegro Hand v4 is a sixteen-DOF tendon-driven anthropomorphic hand: four fingers of four joints each, with the thumb additionally articulated for opposable pinch. The embodiment descriptor declares its kinematic chain via URDF, its actuation envelope (per-joint torque limits, tendon stiffness model), and its tactile sensor placement (four fingertip pressure pads in v0.1 configuration).

Recipe — step-by-step.

Step R1: Approach pose resolution. Compute the pre-grasp pose `T_pre = T_target ∘ Translate(-a, d_clear)` where `d_clear = 15 mm` is the embodiment's published approach clearance. All transforms are stored as 4×4 IEEE 754 binary64 matrices in row-major order. Multiplication is performed by an unrolled 64-multiply, 48-add sequence emitted in fixed source order with FMA explicitly disabled. (*Determinism contribution to I1:* every floating-point operation in this step is a single rounded operation on a fixed operand order; cross-platform byte-equality holds because IEEE 754 binary64 with FMA disabled is bit-identical across conformant hardware.)

Step R2: Embodiment-respect feasibility check. Verify that `T_pre` and `T_target` lie within the Allegro Hand's reachable workspace via the published reachability polytope (a 16-face convex hull stored in the embodiment descriptor). If either pose lies outside the polytope, halt with ⊥`_unsupported`. (*I2 contribution: every emitted action is reachable. I4 contribution: failure is signaled, not silently degraded.*)

Step R3: Pinch geometry resolution. For the Allegro Hand, `grasp_pinch` resolves to thumb-and-index opposing pinch. Compute thumb fingertip target pose `T_thumb` and index fingertip target pose `T_index` such that (i) their midpoint coincides with `T_target`, (ii) they are separated along the pinch axis by `2·r_object` where `r_object` is the object cross-section radius extracted from `T_target`'s metadata, and (iii) their approach normals are anti-parallel to ±`a`. This is a closed-form computation (one square root via a portable polynomial approximation, two cross products); no iterative search.

Step R4: Inverse kinematics for thumb and index. The Allegro thumb has 4-DOF analytical IK admitting closed-form solution; the index has 4-DOF and is solved by 8 iterations of a deterministic Newton method seeded from the embodiment descriptor's `neutral_grasp_pose`. Newton iteration uses a fixed iteration count (not a tolerance threshold) to preserve bit-determinism across platforms — the tolerance achieved is checked post-hoc, and if greater than `1e-9 m`, the recipe halts with ⊥`_unsupported`. (*I1 contribution: fixed iteration count yields fixed-byte output. I2 + I4: tolerance check enforces embodiment-respect, with explicit failure return.*)

Step R5: Tendon-tension to force conversion. The Allegro tendon model relates fingertip normal force to per-tendon tension via a published 4×4 Jacobian (function of joint configuration). Solve for the tendon tension vector achieving a target fingertip force of `F_max / 2` per finger (the budget is split between thumb and index). If any required tension exceeds the per-tendon limit declared in the embodiment descriptor, halt with ⊥`_safety`. (*I4 contribution: explicit safety-envelope failure.*)

Step R6: Trajectory construction. Generate a joint trajectory of `N = 50` waypoints at `Δt = 2 ms` from `neutral_grasp_pose` to the IK solution from R4, linearly interpolated in joint space. Force trajectory follows the tendon-tension setpoint from R5, ramped from zero to target over the final 10 waypoints. (*I1: all interpolation arithmetic is bit-deterministic. I2: per-waypoint joint positions and velocities verified against per-joint velocity limits; any violation halts with* ⊥`_safety` *per I4.*)

Step R7: Tactile-target population. For each fingertip in contact, declare the expected `TactileManifold` pattern: contact point at fingertip pad, force vector normal to surface, magnitude consistent with R5. The tactile target is what the Driver Interface uses to verify execution, and what closed-loop control uses to adapt.

Step R8: Confidence and safety envelope serialization. Populate `confidence = 0.95` (a published constant for grasp_pinch on Allegro in its calibrated operating regime). Inherit the safety envelope unchanged from the Skill Layer invocation. (*I5 contribution vacuously: this primitive uses no extensions; if it did, extension fields would pass through unchanged.*)

Step R9: Canonical action serialization. Emit the populated `CanonicalAction` record using the portable text formatter specified in § 4.2's I1 note: decimal notation with 9 significant figures (sufficient for round-trip exactness of binary64), fields in lexicographic key order, deterministic line terminators. (*I1 contribution: serialization is the final determinism gate; identical inputs through R1–R8 yield byte-identical output here.*)

Invariant satisfaction summary.

Invariant	How this recipe satisfies it
I1 Determinism	Every floating-point operation uses portable binary64 with FMA disabled (R1, R4, R6); fixed iteration counts replace tolerance-based termination (R4); serialization via portable text formatter (R9).
I2 Embodiment-respect	Reachability polytope check (R2); IK tolerance check (R4); tendon-tension limit check (R5); per-joint velocity limit check (R6). Each is a hard precondition.
I3 Composability (rest-stable)	The `grasp_pinch` skill starts from `neutral_grasp_pose` (rest-stable: zero velocity, no held object, no contact) and ends with the object held under closed grip — a rest-stable terminus where `env'` populates the `held_object` field. Boundaries with neighboring rest-stable primitives are therefore I3-testable by construction.
I4 Failure-mode preservation	Three explicit ⊥ return points: ⊥`_unsupported` (R2 reachability, R4 IK convergence), ⊥`_safety` (R5 tendon overload, R6 velocity violation). No degraded outputs.

Invariant	How this recipe satisfies it
I5 Extension-namespace isolation	This primitive uses no extensions; if extensions were present (e.g., `ext.allegro.tendon_recalibration`), R8 would pass extension fields through unchanged to the canonical action, where the Driver Interface either consumes them or returns ⊥`_unsupported` per its declared support.

What this existence proof does and does not establish. What it establishes: a constructively specified retargeting recipe satisfying all five binding invariants exists for at least one non-trivial primitive on at least one real embodiment. The recipe is not a working implementation, but it is concrete enough that an engineer can verify by inspection that no step requires invoking the natural numbers as a witness or hand-waving the floating-point control. What it does not establish: that recipes of similar discipline exist for all fifty primitives (still to be exhibited primitive-by-primitive in the v0.1 reference materials), that the recipe achieves competitive performance against optimized hand-tuned baselines (an empirical question for v1.0), or that the recipe extends to embodiments whose actuation departs sharply from tendon-driven hands (a structural question whose answer requires similar recipes for direct-drive and pneumatic cases, exhibited in the GitHub companion). The existence claim is the minimal claim; everything stronger is left to the reference implementation.

4.5 Worked Example: Cable-and-Connector Insertion

We illustrate the layers together with a worked example drawn from a class of industrial manipulation tasks that motivated the early phase of this work: picking up a flexible cable terminated by a connector and inserting it into a receptacle. The task is structurally non-trivial — it combines compliant-object grasping (the cable is flexible), force-controlled insertion (over-insertion damages pins), and visual servoing (the receptacle pose has tolerance under millimeter scale).

A VLA targeting this task emits a Skill Layer composition:

```
task: insert_cable_into_connector
composition:
  grasp_pinch(target=cable_end_visual, force_budget=3.0N) ;
  reorient(target=connector_alignment_pose) ;
  align(reference=connector_axis, tolerance=2mm) ;
  insert(target=connector_socket, force_budget=8.0N,
         abort_if=force_exceeded) until seated_indicator(force>=4N, depth>=8mm)
```

The Translation Layer expands this composition into a sequence of canonical actions, each parameterized for the target embodiment. For a four-finger tendon-driven hand (e.g., Allegro), `grasp_pinch` resolves to a thumb-and-index opposing grasp with joint trajectory derived from the task-frame target pose; the safety envelope's 3N force budget is expressed as bounded joint torques. For a four-finger direct-drive hand (e.g., LEAP), the same skill resolves to the same opposing geom-

etry but with different torque expressions reflecting the embodiment's actuation properties. For a six-finger pneumatic hand (e.g., Bridgestone TETOTE), the skill resolves to a front-two-finger pinch with the force budget expressed as a pneumatic pressure setpoint (approximately 0.3 MPa for a 3N target force, calibrated through the embodiment descriptor).

Each resolved canonical action is dispatched to the embodiment's ROS 2 action server. The Driver Interface translates the canonical action into motor commands appropriate to the embodiment's actuation. Tactile feedback flows back through the Translation Layer, where it is converted into the TactileManifold representation and made available to the VLA for closed-loop control.

The crucial observation: the VLA's composition is identical across all three embodiments. The VLA does not need to know whether its target is tendon-driven, direct-drive, or pneumatic; it does not need to know whether the hand has four fingers or six. The translation is performed by the layer, not by the model. This is the property that, multiplied across the realistic deployment matrix described in § 1.1, collapses the N $\times$ M integration cost to additive.

A note on where this composition originates, since it carries the standard's central dependency assumption. The Skill Layer composition above is emitted by the planner tier: the hierarchical-VLA, Code-as-Policies, or "System-2" reasoning layer that interprets a task and produces structured sub-goals. It is not emitted by an action-token policy that outputs low-level continuous motor commands at tens of hertz. Those action-token VLAs (π0, OpenVLA, GR00T) sit below the canonical-action boundary, close to the Driver Interface, and RFL does not ask them to produce symbolic compositions. The natural emitter of the Skill ISA is the general planner archetype that a frontier model already provides. That this is realizable rather than aspirational is established by the project's demand-side existence proof, in which a current frontier model, given the Skill ISA schema but not the embodiment, emits valid full-spec Skill ISA on every task in a held-out set of novel manipulation problems. RFL therefore rests on the planner tier producing structured intent, which is the function of that tier, and not on action-token models acquiring a new output modality.

A second observation: the composition's `insert` step uses the Reactive operator (`until seated_indicator(...)`), which allows the action to terminate cleanly when tactile feedback indicates seating. The Skill Layer's compositional algebra makes this reactive behavior expressible without requiring the VLA to micromanage timing — the embodiment-side implementation of `seated_indicator` is part of the Driver Interface, and its specific implementation varies by embodiment (force-threshold-based for one, deformation-pattern-based for another) but its semantics are uniform.

4.5.1 I3 decomposition: where the composability invariant does and does not apply

The cable-insertion composition exposes both the kind of boundary at which I3 holds and the kind at which it does not. We work through one of each.

A rest-stable boundary: between `grasp_pinch` and `reorient`. After `grasp_pinch` completes, the embodiment holds the cable end under closed grip; if we *introduce* a `stabilize` primitive at this point (the Skill Layer encourages this as a discipline), the boundary becomes rest-stable in the sense

of Definition I3.1:

1. All joints at zero commanded velocity at the stabilize-terminal frame (✓ — `stabilize` is rest-to-rest by construction).
2. Active grasp present (× on the literal Definition I3.1 reading) — but the active grasp is *part of the environment state passed to `reorient`*: `env'` encodes "cable held in pinch grip, pose X." `reorient`'s retargeting is parameterized by this `env'` and produces a trajectory that begins from the held-grasp configuration. The grasp is preserved across the boundary as a discrete state, not as continuous force; the I3.1 zero-grasp condition is met when interpreted as "no in-hand object whose continuous dynamics are not captured by `env'`." The Skill Layer formalizes this through the `held_object` field of `EnvironmentState`.
3. No active contact forces beyond the static grip required to hold the cable (✓).
4. Environment quiescent: cable end held still, receptacle at fixed pose, no other commanded motion (✓).

Under these conditions, the retargeter for the four-finger Allegro hand satisfies:

$$
\begin{aligned}
&\texttt{retarget(grasp\_pinch; stabilize; reorient, Allegro, env}_0\texttt{)} \\
&\quad\equiv\ \texttt{concat(} \\
&\qquad \texttt{retarget(grasp\_pinch, Allegro, env}_0\texttt{),} \\
&\qquad \texttt{retarget(stabilize, Allegro, env}_1\texttt{),} \\
&\qquad \texttt{retarget(reorient, Allegro, env}_2\texttt{)} \\
&\quad\ \texttt{)}
\end{aligned}
$$

byte-for-byte, where $\texttt{env}_1$ and $\texttt{env}_2$ are the environment states after `grasp_pinch` and `stabilize` complete. The Class 2 conformance suite tests this equality mechanically.

A non-rest-stable boundary: between `align` and `insert`. The boundary between `align` and `insert` is *deliberately* non-rest-stable in the composition as written. `align` terminates with the connector approaching the socket along its axis, carrying terminal velocity (the alignment converges asymptotically), still in contact with no rest condition. An efficient retargeter for the joint skill `align; insert` exploits this terminal velocity to start `insert` with the correct initial impedance and approach velocity. Decomposing the retargeting at this boundary would either (i) require `align` to terminate at zero velocity — adding 200–400 ms of cycle time and losing the kinematic continuity that improves insertion success — or (ii) require the independent retargeting of `insert` to re-derive the approach velocity from `env'`, which is feasible but produces a *different* canonical action than the joint retargeting would produce. Byte-equality fails by design.

The spec handles this by *not* requiring I3 across this boundary. The Skill Layer's `align; insert` pattern is a recognized "fused" pair; the v1.0 reference retargeter is permitted to retarget the pair as a single unit and the Class 2 suite excludes the `align`→`insert` boundary from its I3 test corpus. A VLA that needs the rest-stable form can write `align; stabilize; insert` instead, at the cost of cycle time, and recover I3 testability at that explicit cost.

What the worked example demonstrates. I3 in the scoped form is a real, testable property of the

retargeting interface — not a wish. It is also a *partial* property: it holds at boundaries where the Skill Layer composer has chosen rest-stable composition, and does not hold elsewhere. The Skill Layer is designed to make the choice between "compose with rest-stable boundaries (I3 testable)" and "compose with fused/non-rest-stable boundaries (efficient but not I3-decomposable)" a visible decision in the source composition rather than a hidden property of the retargeter. The cable-insertion example uses both modes deliberately, in their natural places.

This worked decomposition is one of several that the v0.1 conformance test corpus exercises; the GitHub reference implementation will ship with the full corpus covering each of the seven primitive categories and each of the five composition operators, with each composition explicitly annotated as I3-testable or I3-exempted at the boundary level.

4.6 Conformance and Certification

A specification without a conformance regime is a recommendation, not a standard. RFL ships v0.1 with an open-source conformance test suite that defines, mechanically, what it means for an implementation — VLA or embodiment — to be RFL-compliant.

The suite has four test classes.

Capability conformance. For every primitive an embodiment declares in its capability manifest, the suite executes a battery of test invocations against a calibrated test bench and measures success rate, latency, and safety-envelope adherence. The empirical results become the values populating the manifest; an embodiment's claim of `success_rate: 0.95` is grounded in measurement, not assertion. The test bench specifications are themselves public, so that independent labs can reproduce conformance results.

Composition conformance. The suite executes a fixed library of multi-primitive compositions (sequential, conditional, concurrent, reactive, retry) and verifies that the embodiment's behavior remains within the declared safety envelope of the composition and that the composition's outcome matches the declared semantics. A composition that succeeds on the individual primitives but produces a safety-envelope violation in composition fails the conformance test, regardless of the individual primitive results.

Capability discovery conformance. The suite queries the capability manifest, then attempts primitives that the manifest does and does not declare; the embodiment must succeed on declared primitives at the declared rate and must cleanly reject (not silently fail, not unsafely attempt) primitives it does not declare.

Safety conformance. The suite deliberately invokes primitives at the edge of and beyond their declared safety envelopes (in simulation; certain physical tests are run only on instrumented test articles). The embodiment must abort within the declared response time and must leave the system in a safe state after the abort.

An implementation passing all four classes at a declared spec version is *RFL-compliant* at that version and may use the RFL™ trademark on packaging and documentation per the RFL™ trademark

policy (a governance-level decision, out of scope for this specification). Implementations passing partial classes are *RFL-conformant-subset* and may declare which classes they pass, but may not use the trademark.

The conformance regime is what makes the Certification reinforcement loop (§ 5.4 L4) and the Insurance reinforcement loop (§ 5.4 L8) operational. A notified body certifying a deployment against ISO 10218 or ISO 13482 has a defined evaluation surface — the RFL conformance results plus the embodiment's deployment context — rather than having to evaluate the deployment from first principles. An insurer pricing liability against an RFL-compliant deployment has a defined risk profile — the empirical conformance results across the deployed primitives — rather than relying on per-deployment underwriting.

---

5 § 5. Comparative Analysis

A proposal to occupy a structural position is most credibly evaluated by examining the positions that have been occupied before and the ones that have remained empty despite attempts. This section places RFL alongside three historical coordination layers chosen for their structural relevance — ARM, CUDA, and USB — and the contemporary cohort that adjacent to RFL but does not occupy its position. The aim is not to claim RFL will replay any of these histories, but to identify the structural dimensions on which RFL must satisfy the conditions that prior successes satisfied and avoid the conditions that prior failures encountered.

5.1 The Framework

Coordination layers vary along three dimensions that matter for long-run defensibility. The first is *technical position*: the layer of the stack at which the abstraction is drawn, the side or sides of the market it abstracts, and the kind of object it standardizes (instruction, message, data format, action). The second is *governance form*: who owns the specification, who controls its evolution, how participation is regulated. The third is *constitutional discipline*: which commitments the steward refuses to abandon under commercial pressure, and which structural mechanisms enforce that refusal.

Each historical case occupies a distinct combination on these dimensions, and the combinations have proved more or less durable. The analysis below maps RFL against three of them in turn.

5.2 ARM: The Structural Correspondence

ARM is the closest structural analog and the case from which RFL's design pattern is most directly derived. § 3.3 stated the eight-element correspondence between ARM ISA design choices and RFL specification choices. § 5 emphasizes the dimensions on which ARM's history offers operational lessons for RFL's trajectory.

Three specific moments in ARM's history bear directly on choices RFL faces. First, ARM's decision in 1990 to spin out of Acorn as an independent entity — sacrificing the captive customer that

funded its research to gain the neutrality required for cross-vendor adoption. The parallel for RFL is the choice to organize as a Linux Foundation subsidiary rather than as a wholly-owned project of any Founding Member, and to bind the Foundation's revenue model in ways that prevent vertical capture. The structural lesson: neutrality has a price paid up-front and a value recovered over decades.

Second, ARM's near-death experience in the desktop market from 2000–2005, when Intel's superior process technology threatened to make ARM irrelevant outside embedded niches. ARM's survival came from doubling down on the mobile market — accepting that desktop dominance was unwinnable and pivoting toward the segment where its energy-efficiency advantage compounded into a structural lead. The parallel for RFL is the imperative to choose the segment where the N×M crisis is most acute and the alternative (vertical integration) is most painful — likely the industrial manipulation and humanoid-services segments where the embodiment diversity is highest and no single VLA provider has yet locked in dominant share. The structural lesson: coordination layers win the segment where vertical integration is least tolerable, not necessarily the segment of largest absolute size.

Third, ARM's response to the v8 → v9 architectural transition (2018–2022), which preserved binary compatibility for the v8 instruction base while adding new architectural features. ARM's discipline in not breaking compatibility, even when commercial parties advocated for it, protected the multi-decade application base on which its position rested. RFL's forward-compatibility principle (§ 3.2 Principle 5) is the descendant of this discipline; the operational instantiation lives in the project's spec evolution procedure (out of scope for this specification).

ARM's history demonstrates that the structural position RFL seeks is occupiable, that its defensibility is durable, and that the commercial pressures against neutrality are real and recurring. ARM's history does not, however, demonstrate that physical-AI specifically will follow the same trajectory; that question is settled by execution.

5.3 CUDA: The Cautionary Inverse

CUDA occupies the cautionary inverse of ARM's position. It is a coordination layer in a technical sense — application code targeting CUDA runs across NVIDIA's GPU product line and benefits from the network of optimized libraries (cuBLAS, cuDNN, TensorRT) — but it is owned and operated by a single vendor whose primary commercial interest is selling the underlying hardware. The result is a layer of extraordinary technical sophistication that has nevertheless attracted, over the past three years, an aggressive industry-wide effort to escape it (the Triton language from OpenAI, the MAX platform from Modular, Intel's oneAPI, AMD's ROCm, Apple's Metal Performance Shaders, and the emerging cohort of inference compilers targeting non-NVIDIA accelerators).

The CUDA case shows what RFL must not become. A coordination layer whose steward also competes in adjacent markets generates a structural incentive for adopters to seek alternatives the moment the alternatives become technically viable. NVIDIA's commercial response — adding more value at the CUDA layer faster than the alternatives can erode adoption — is sustainable while

NVIDIA's technical lead in silicon is unassailable, but it is not a structural defense; it is a continuously running engineering campaign. RFL's neutrality commitment — the first of the constitutional commitments the project's stewardship layer is intended to bind itself to — is the structural alternative to this campaign.

The CUDA case also shows what is at stake in the choice. Had NVIDIA chosen to spin CUDA out into a neutral foundation in 2014, when its position was strong enough to dictate terms, the GPU computing ecosystem would today have a single dominant abstraction layer rather than the fragmenting cohort of alternatives that the captive ownership produced. The opportunity that NVIDIA declined is the opportunity RFL is positioned to take in physical AI before the equivalent verticalization ossifies.

5.4 USB: An Alternative Governance Model

USB occupies a third position relevant to RFL's design: it is a consortium-governed standard (the USB Implementers Forum, a non-profit), narrower in scope than either ARM or CUDA, but with notably broader cross-vendor adoption than either. The USB-IF model — a member-funded consortium that owns the specification, certifies compliant implementations, and licenses the trademark — is the closest existing governance model to the form intended for the RFL project's stewardship. (The governance design itself is out of scope for this specification; the companion position paper develops it.)

Three features of USB-IF's structure are directly adopted by RFL. First, the trademark gate: USB devices cannot legally display the USB logo without passing certification, and this commercial-grade legibility of conformance has proved more effective at enforcing compatibility than any technical mechanism. RFL adopts the same pattern; the trademark policy lives in the project's governance document, out of scope here. Second, the working-group structure: USB-IF's technical work is organized into focused working groups with rotating chairs from member organizations, which prevents single-organization dominance and ensures multiple perspectives shape each spec revision. RFL's working-group structure follows the same template; the working-group composition is a governance-level decision out of scope here. Third, the modular extension mechanism: USB has evolved from 1.0 through 4.0 over thirty years by adding capabilities without breaking compatibility, governed by the same body throughout. RFL's extension registry (§ 6.1) is structurally similar.

Two features of USB-IF's structure are deliberately not adopted. USB-IF's membership dues are dominated by a small number of large hardware companies, which has produced occasional concerns about asymmetric influence over spec direction; The RFL project's planned three-tier membership structure is designed to dilute this risk; the membership design lives in the governance document, out of scope here. And USB-IF has no equivalent of the constitutional commitments planned for the RFL stewardship layer; the absence has not been problematic in USB's history but is a deliberate strengthening in a domain where the long-run governance pressures are more intense.

5.5 The Contemporary Cohort

The contemporary efforts — ROS 2, LeRobot, Open X-Embodiment, and the NVIDIA Isaac/GR00T stack — were treated in § 2 as adjacent efforts that do not occupy the RFL position. From the comparative-analysis perspective, the additional observation worth making is that each occupies a different structural position from the others, and the differences themselves are evidence that the physical-AI coordination problem has been recognized but not resolved.

ROS 2 is governance-strong (Open Source Robotics Foundation, multi-stakeholder) but technical-position-narrow (transport layer, not action semantics). LeRobot is technical-position-broader but governance-weak (Hugging Face captive). Open X-Embodiment is technical-position-narrow (training data, not runtime) and governance-asymmetric (Google-dominated). NVIDIA Isaac is technical-position-broadest of the four (full stack) but governance-foreclosed (vertically integrated). Each pair of strengths is missing the complementary strength that the structural position requires. RFL is designed to combine ROS 2's governance discipline with Isaac's technical scope, neither of which exists in isolation.

5.6 The Position That Emerges

Set the analyses together and the position RFL is designed to occupy resolves to a specific combination:

Dimension	ARM	CUDA	USB	RFL
Technical layer	ISA	Compute API	Bus	**Action semantics + embodiment translation**
Sides abstracted	OS ↔ silicon	App ↔ GPU	Host ↔ device	**VLA ↔ embodiment**
Governance form	Single company (neutral)	Single vendor	Consortium	**LF-hosted foundation**
Constitutional binding	High (de facto)	None	Medium	**High (de jure)**
Verifiable safety	n/a	n/a	n/a	**Yes (§ 4.5)**
Trademark gate	License-mediated	Vendor-controlled	Certification-gated	**Certification-gated**
Cross-vendor adoption	Very high	Declining	Very high	**Target: very high (unrealized at v0.1)**

The combination is novel along two axes. First, RFL pairs ARM-class technical depth with USB-class consortium governance — a combination neither historical case achieved (ARM was governance-light, USB was technically narrow). Second, RFL introduces verifiable safety semantics as a first-class architectural concern, which none of the historical cases needed to address because none of them mediated physical action.

The empty position in the historical landscape is therefore not a contingent gap but a structural one: it exists because no prior coordination problem combined the cross-vendor abstraction need with the physical-safety need at the layer where the abstraction is drawn. The physical-AI coordination problem is the first to do so, and the abstraction layer that resolves it must combine governance and verification disciplines that have not previously been combined. The Foundation, the specification, and the principles described in §§ 3–7 are the design that combines them.

---

6 § 6. Implementation Roadmap

The specification of § 4 is presented at version 0.1: the architectural commitments are intended to be stable, while schema details and the extension registry remain open during the v0.1 review period. This section outlines the path from v0.1 to the v1.0 freeze (2027 Q4 target) and the conformance regime that surrounds it.

6.1 Spec Maturation

Three planned major versions on the horizon of this paper:

- **v0.1 (2027 Q1)**: Initial reviewable release. Architectural commitments — the three-layer division, the fifty primitives and their categorization, the compositional algebra, the canonical action tuple, the retargeting interface contract and its five binding invariants, the TactileManifold abstraction, the four conformance test classes — are fixed at release. Schema details, error codes, and the extension registry remain open to community comment through a 90-day public review.
- **v1.0 (2027 Q4)**: First stable specification. Incorporates v0.1 review feedback. From v1.0 onward, the forward-compatibility principle (§ 3.2 Principle 5) takes binding effect: no breaking changes within the v1.x line. v1.0 is the version against which initial production deployments are certified, and ships with the first reference implementation of the retargeting algorithm satisfying the v0.1 interface contract.
- **v2.0 (2029, projected)**: First major-version evolution. Anticipated to incorporate requirements visible in 2026 but not yet stable enough to specify in v1.0: lifelong-learning hooks for VLAs, soft-robotics actuation extensions, cross-modal grounding interfaces between RFL skills and ambient world-model representations. v2.0 preserves backward compatibility with the v1.x line through a versioned extension mechanism analogous to ARM's architecture-profile evolution.

A separate evolution track manages the **extension registry** — a public catalog of optional, namespaced extensions that vendors or vertical-specific working groups can propose without modifying the core spec. Capabilities too specific to belong in the core, or too novel to commit to before field validation, enter the registry and may be promoted to the core in subsequent major versions if they accumulate cross-vendor adoption.

6.2 Reference Implementation

The reference implementation is the artifact through which the specification is exercised, demonstrated, and validated. It is open-source under Apache 2.0 and is intended as the canonical-by-construction definition of the retargeting interface contract (§ 4.2): any divergence between the reference and a third-party implementation on a Class 2 (determinism) conformance test is, by convention, a defect of the third-party implementation rather than of the reference.

- **Core (Rust).** The Skill Layer parser, the Translation Layer retargeting algorithm, and the Driver Interface protocol implementation are written in Rust. The choice is governed by three requirements: memory safety in a system that mediates physical action, deterministic latency suitable for real-time integration, and a foreign-function interface that exposes cleanly to both Python (for ML integration) and C (for embedded firmware).
- **Python bindings.** Generated from the Rust core through PyO3, providing the ML-research-facing API. The Python layer composes naturally with PyTorch and JAX pipelines. A LeRobot-compatible adapter ships as a v0.1 extension.
- **C bindings.** Generated from the Rust core through cbindgen, providing the embedded-firmware-facing API. The C layer compiles cleanly under the constrained toolchains common in robot controller firmware.

Initial support matrix (v0.1 launch, 2027 Q1). Three VLAs and three embodiments, paired in all nine combinations:

- VLAs: Physical Intelligence $\pi 0$, Stanford OpenVLA, and a Hugging Face SmolVLA variant
- Embodiments: CMU LEAP Hand, Wonik Allegro Hand, and a third embodiment of structurally distinct actuation class (e.g., a pneumatic six-finger hand) drawn from the initial implementer cohort

The nine-cell support matrix is the initial demonstration that the spec is genuinely cross-vendor on both sides. The matrix expands at v1.0 and at v2.0.

6.3 Conformance Ecosystem

Conformance — the mechanism by which an implementation can demonstrate it meets the spec — matures through three distinct tiers.

Tier 1 (2027 Q1–Q3): self-certification. Implementers run the open-source conformance test suite on their own hardware and publish the results. A public registry of declared conformance results is maintained but not independently verified at this tier. Integrity rests on the reproducibility of the open-source test suite and the reputational cost to any implementer who publishes false results.

Tier 2 (2027 Q4–2028 Q2): steward verification. As implementer count grows, the project's stewarding body establishes a verification capability operated by full-time engineering staff. Steward-verified conformance receives a distinguished badge in the public registry and is the gate for use of the RFL™ trademark on packaging.

Tier 3 (2028 Q3 onward): third-party notified bodies. As the conformance regime matures and the regulatory context demands independent assessment, third-party notified bodies operate the verification process. This is the configuration that enables ISO 10218 / 13482 certification of RFL-compliant deployments to flow through the conformance result rather than through per-deployment re-evaluation.

6.4 Stewardship Commitments

A specification's credibility as a coordination layer rests not only on its technical content but on the predictability of the entity that stewards it. We sketch the binding stewardship commitments here at the level required to evaluate the specification itself; the full institutional and governance framing is the subject of the companion position paper at the project page, and a separate Governance Charter is planned for publication concurrently with v0.1.

The commitments below are intentionally compact and constitutional in form — five short clauses, each of which is verifiable from public artifacts. They are listed here because the spec stands or falls on them: a reader cannot evaluate "is RFL a coordination layer I should adopt?" without knowing what the stewarding entity is bound to do and not do.

C1 — Provider neutrality (binding from v0.1). The stewarding entity does not produce VLAs, does not produce embodiments, and does not produce commercial applications built on RFL. Revenue, if any, is restricted to trademark licensing, conformance verification fees, and grant funding. No VLA vendor and no embodiment vendor controls a majority of any technical or governance decision-making body.

Operational mechanism (binding from v0.1). "Effective governance share" is defined per-body as the sum, over all positions on that body, of each member organization's weighted seat count, where weights derive from the membership tier (Anchor: 1.0; Contributing: 0.5; Observer: 0). For each constituted body (Technical Steering Committee, Conformance Committee, Extension Registry Committee, Board), the effective shares are audited *quarterly* by an independent auditor whose appointment is rotated every two years and ratified by the Board. The audit report is published within thirty days of completion.

The 30% rebalancing process. If any single organization's effective share on any single body exceeds 30% in a published audit, a six-month rebalancing window opens. During the window, the affected body may not adopt binding decisions that the over-share organization could plausibly benefit from disproportionately (the specific decision categories triggering this restriction are enumerated in the Governance Charter). The over-share organization is expected to either reduce its participation in that body to below 30% or to recruit additional Anchor-tier members of equivalent weight from non-correlated organizations within the window. If neither occurs by the end of the window, the Board is bound to dilute the over-share by increasing the Anchor seat count from non-correlated organizations until the share falls below 30%; the increased seat count persists for the remainder of the two-year governance term. The mechanism is mechanical rather than discretionary precisely because the failure mode it guards against — gradual single-vendor capture — historically pro-

ceeds through individually-reasonable incremental decisions that no single discretionary review would catch.

Three-tier membership. The Anchor tier is reserved for organizations making the largest binding commitments (financial, engineering, and conformance) and carries full voting weight. The Contributing tier accommodates organizations participating in technical work without the financial commitment Anchor membership requires; it carries half voting weight. The Observer tier accommodates academic, regulatory, and civil-society participants; it carries voice but no vote. Tier assignment is reviewed annually; promotion and demotion criteria are published. The dilution mechanism above operates on Anchor-tier seats specifically because Anchor weight is what drives the 30% threshold; the Contributing and Observer tiers are not subject to the 30% cap because they cannot individually drive a decision.

C2 — Permissive licensing (binding from v0.1). The specification, reference implementation, conformance test suite, and all stewarded artifacts are released under Apache License 2.0 or a license with materially equivalent commercial-use, modification, and redistribution permissions. License downgrade is not permitted; future major versions may only relax, not tighten, the licensing terms.

C3 — Scope discipline (binding from v0.1). The stewarding entity maintains and produces only: the specification, the reference implementation, the conformance test suite, the extension registry, and supporting documentation. The entity does not enter adjacent markets (training-tool products, deployment platforms, integration services) in competition with implementers. Scope is published; scope expansion requires a public charter amendment.

C4 — Transparency (binding from v0.1). Technical decision-making (specification changes, extension acceptance, conformance criteria) occurs in public proceedings with published minutes and rationale. Conformance test results are published in a public registry. Closed sessions are permitted only for security-disclosure and personnel matters; categories and rationales for closed sessions are themselves public.

Operational mechanism (binding from v0.1). Each constituted body publishes its meeting agenda at least 72 hours in advance, holds the meeting on a webcast that is recorded and archived, and publishes minutes (including dissents) within ten business days. Spec-affecting decisions follow a Request-for-Comment (RFC) process: the proposing organization files a public RFC with a minimum 30-day comment window, the relevant committee resolves the RFC in a recorded session, and the resolution (acceptance, modification, or rejection with rationale) is published in the public RFC archive. The conformance registry exposes per-implementation, per-class results with sufficient detail that an outside party can re-run the equivalent test suite and verify reproducibility; results that cannot be independently reproduced are flagged as such in the registry rather than removed.

C5 — Welfare primacy (binding from v0.1). Specification decisions that materially trade off safety, accessibility, or other welfare-relevant properties are resolved in favor of welfare, with the rationale documented. The stewarding entity maintains an independent welfare review process whose decisions can override technical or commercial recommendations on welfare grounds, subject to published procedures.

These commitments are stated as constitutional clauses because reversibility is the property they primarily protect against. The expected institutional form is a Linux Foundation subsidiary or equivalent neutral nonprofit, but the choice of institutional vehicle is secondary to the clauses themselves — a Linux Foundation subsidiary that operated against these clauses would fail the same way an independent foundation that operated against them would. The clauses, not the institutional choice, are load-bearing.

The companion position paper develops the historical evidence (ARM Holdings, USB-IF, Khronos, OpenSSF) for why these specific clauses, in this combination, suffice to make a horizontal coordination layer credible enough to attract cross-vendor adoption. We refer interested readers there.

6.5 Cold-Start: How an Implementer Adopts v0.1

A standard with no captive customer faces the cold-start problem in its sharpest form: every individual implementer's adoption is rational only if other implementers also adopt, and the first implementer's adoption is rational only if a credible commitment exists that the second will follow. ARM dissolved this in 1990 by spinning out of Acorn with Acorn itself as the captive first customer. RFL has no captive first customer and the spec acknowledges this is a substantive risk. We sketch here the minimal strategy the spec assumes, at the level needed for an evaluator to judge whether the spec's other claims are defensible against the cold-start question; the full incentive design is in the companion position paper.

The strategy has three components, sequenced.

Component 1 — Adoption asymmetry on the embodiment side. Embodiment manufacturers have stronger incentives to adopt than VLA providers do at v0.1, because (i) every additional VLA an embodiment supports is a revenue surface, (ii) embodiment manufacturers compete on the multiplier between hardware sales and software capability, and (iii) the cost of adding RFL conformance to an embodiment already targeting ROS 2 is incremental, not architectural. The strategy targets three embodiment manufacturers from structurally distinct actuation classes (one tendon-driven hand, one direct-drive hand, one humanoid platform) as the v0.1-launch implementer cohort. Their incremental cost is low; their published support of RFL becomes a competitive differentiator against non-conforming competitors.

Component 2 — One VLA provider whose business model aligns. The asymmetric incentive cuts the opposite direction for vertically integrated VLA providers (Figure, Tesla, NVIDIA), who benefit commercially from cross-VLA fragmentation. The relevant first VLA adopter is an organization whose business model *does not* depend on lock-in: a research lab releasing models under permissive licenses (Hugging Face's SmolVLA line, Stanford's OpenVLA, or an analogous group). These organizations adopt for reach, not exclusivity, and their adoption is what catalyzes the multi-VLA $\times$ multi-embodiment matrix that makes the standard binding.

Component 3 — A reference deployment that is independently valuable. The nine-cell support matrix (§ 6.2) is structured so that the matrix itself, independently of RFL's broader claims, is a usable artifact: three VLAs deployable across three embodiments with a single integration cost amortized

across the matrix. An evaluator considering whether to adopt v0.1 against the alternative (per-pair integration) can compute the saved integration cost directly, even before assigning any weight to the long-run network-effect claims.

The strategy does not guarantee cold-start success. It commits the spec to a specific posture: the first cohort is recruited from organizations whose business model makes adoption rational under conservative assumptions about everyone else's behavior. If three embodiment manufacturers and one open VLA provider commit by 2027 Q1, the structural argument for v0.1's coordination value is realized. If they do not, the spec at v0.1 represents an unrealized coordination opportunity and the next major version must reconsider both the spec content and the adoption strategy. The cold-start question is, by its nature, not resolvable by argument from within the spec; it is resolved or not by the first wave of implementer commitments. The spec's responsibility is to be designed so that those commitments are rational to make, which is what the technical and stewardship choices of this document attempt to do.

6.6 What Is Out of Scope

Commercial product development above the open specification — applications, training tools, deployment platforms — is not part of this roadmap. The roadmap describes what the project produces as the steward of the open spec, plus the conformance ecosystem that surrounds it.

Geographic expansion is also absent. The project operates from one initial jurisdiction and recognizes compliance regimes across all jurisdictions through partnerships with relevant national bodies.

Acquisition or merger of adjacent projects is absent. The project will collaborate with LeRobot, Open X-Embodiment, ROS 2, and other adjacent efforts through technical liaison rather than corporate consolidation, in accordance with the Provider Neutrality principle (§ 3.2 Principle 4 and Stewardship Commitment C1 above).

---

7 § 7. Conclusion

7.1 What This Paper Is and Is Not

Before concluding, we mark explicitly what this paper does and does not establish, because we believe the distinction matters more to whether v0.1 is taken seriously than any individual passage of the spec.

This paper is a *coordination-layer specification proposal* at the architectural-commitment level: the three-layer decomposition, the fifty-primitive Skill ISA, the retargeting interface contract with five binding invariants, the TactileManifold abstraction, the split conformance regime, the constitutional stewardship commitments. Each commitment is concrete enough that a downstream implementer or critic can verify by inspection whether their proposed alternative is consistent or in

conflict.

This paper is *not* a working reference implementation, and that gap is its load-bearing weakness. A coordination-layer proposal whose contract has been defined but whose first instance has not been built is a proposal whose chief unanswered question is whether the contract is satisfiable in practice, not whether it is well-specified on paper. The § 4.4 Reference Retargeting Recipe (grasp_pinch on Allegro Hand) gives an *algorithmic-specification-level* existence proof — a recipe that a competent engineer can verify by inspection satisfies I1–I5 — but algorithmic specifications are not the same as running code, and the difference between the two is roughly the difference between "plausible" and "demonstrated."

We acknowledge this directly because we judge that **further textual refinement of this paper yields rapidly diminishing returns relative to building the first running implementation**. The structural argument of §§ 1–2 has reached the point where additional polish does not add information, the contract design of § 4 has reached the point where additional restatement does not add precision, and the stewardship commitments of § 6.5 have reached the point where additional elaboration without a sponsoring institutional vehicle is suspended in air. The next-most-valuable artifact this project can produce is not v0.1 + ε of the spec; it is v0.0.1 of the reference implementation, exercised end-to-end against one (VLA, embodiment) pair, with the measured integration cost published. The v0.1 review period that this paper opens is designed to run in parallel with that engineering work; the paper's contribution to the review is the framing, not the demonstration.

Readers who would adopt or invest against v0.1 should accordingly weight the reference-implementation milestone (planned 2027 Q2–Q3) more heavily than the spec text itself. The spec text establishes that the design exists and is internally consistent; the reference implementation will establish that the design is constructible and externally useful.

---

8 References

8.1 A. Foundational network-effect and platform theory

1. Rochet, J.-C., & Tirole, J. (2003). Platform Competition in Two-Sided Markets. *Journal of the European Economic Association*, 1(4), 990–1029. [verified 2026-05-29]
2. Crémer, J., & McLean, R. P. (1985). Optimal Selling Strategies under Uncertainty for a Discriminating Monopolist When Demands Are Interdependent. *Econometrica*, 53(2), 345–361. [verified 2026-05-29]
3. Crémer, J., & McLean, R. P. (1988). Full Extraction of the Surplus in Bayesian and Dominant Strategy Auctions. *Econometrica*, 56(6), 1247–1257. [verified 2026-05-29]
4. Chen, A. (2021). *The Cold Start Problem: How to Start and Scale Network Effects*. Harper Business (HarperCollins imprint). ISBN-13: 9780062969743. [verified 2026-05-30]

5. Helmer, H. (2016). *7 Powers: The Foundations of Business Strategy*. Deep Strategy LLC. ISBN 9780998116310. [verified 2026-05-29]

6. Eisenmann, T., Parker, G., & Van Alstyne, M. (2006). Strategies for Two-Sided Markets. *Harvard Business Review*, 84(10), October 2006. (Reprint R0610F.) [verified 2026-05-29]

7. Hancock, J. T., Naaman, M., & Levy, K. (2020). AI-Mediated Communication: Definition, Research Agenda, and Ethical Considerations. *Journal of Computer-Mediated Communication*, 25(1), 89–100. DOI: 10.1093/jcmc/zmz022. [verified 2026-05-29]

8. Hara, Y. (2026). *Network Effects: Foundational Theory through Implementation Transition*. Internal compilation, 44 chapters / 1.2 MB. Relevant chapters cited inline as "Hara, 2026, ch. N." [Working paper — public release status to be determined; see § 5 footnote on citation conventions]

8.2 B. Robotics theory: grasping, manipulation, and design taxonomy

9. Feix, T., Romero, J., Schmiedmayer, H.-B., Dollar, A. M., & Kragic, D. (2016). The GRASP Taxonomy of Human Grasp Types. *IEEE Transactions on Human-Machine Systems*, 46(1), 66–77. (33 grasps reducible to 17 hand configurations.) [verified 2026-05-29]

10. Cutkosky, M. R. (1989). On Grasp Choice, Grasp Models, and the Design of Hands for Manufacturing Tasks. *IEEE Transactions on Robotics and Automation*, 5(3), 269–279. [verified 2026-05-29]

11. Topkis, D. M. (1998). *Supermodularity and Complementarity*. Princeton University Press. ISBN 9780691032443. Part of "Frontiers of Economic Research" series; 272 pages. [verified 2026-05-29]

8.3 C. Theory of Mind and machine reasoning about other agents

12. Premack, D., & Woodruff, G. (1978). Does the chimpanzee have a theory of mind? *Behavioral and Brain Sciences*, 1(4), 515–526. [verified 2026-05-29]

13. Baker, C. L., Saxe, R., & Tenenbaum, J. B. (2009). Action understanding as inverse planning. *Cognition*, 113(3), 329–349. DOI: 10.1016/j.cognition.2009.07.005. [verified 2026-05-29]

14. Goodman, N. D., & Frank, M. C. (2016). Pragmatic Language Interpretation as Probabilistic Inference. *Trends in Cognitive Sciences*, 20(11), 818–829. (Rational Speech Act framework.) [verified 2026-05-29]

15. Rabinowitz, N. C., Perbet, F., Song, H. F., Zhang, C., Eslami, S. M. A., & Botvinick, M. (2018). Machine Theory of Mind. *Proceedings of the 35th International Conference on Machine Learning (ICML 2018)*, PMLR vol. 80, pp. 4218–4227. arXiv:1802.07740. (ToMnet meta-learning architecture.) [verified 2026-05-29]

16. Kosinski, M. (2024). Evaluating large language models in theory of mind tasks. *Proceedings of the National Academy of Sciences*, October 2024. DOI: 10.1073/pnas.2405460121. (Earlier

preprint: arXiv:2302.02083; GPT-4 reached 75% on 40 false-belief tasks, matching 6-year-old children.) [verified 2026-05-29]

8.4 D. VLA / Foundation models for robotics

17. Brohan, A., Brown, N., Carbajal, J., Chebotar, Y., Chen, X., et al. (2023). RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control. arXiv:2307.15818. (54 authors total.) [verified 2026-05-29]

18. Open X-Embodiment Collaboration (2023). Open X-Embodiment: Robotic Learning Datasets and RT-X Models. arXiv:2310.08864 (current version v9, 2025-05-14). (293+ contributors from 21 institutions, 22 embodiments, 527 skills, 160,266 tasks.) [verified 2026-05-29]

19. Black, K., et al.; Physical Intelligence (2024). $\pi 0$: A Vision-Language-Action Flow Model for General Robot Control. arXiv:2410.24164. (Trained on 7 robotic platforms and 68 unique tasks.) [verified 2026-05-29]

20. Physical Intelligence (2025). $\pi 0.5$: A VLA with Open-World Generalization. *Public blog post / technical communication*. https://www.pi.website/blog/pi05 [verified 2026-05-29 via WebFetch of pi.website; formal arXiv preprint, if any, to be confirmed at editorial polish]

20a. Physical Intelligence (2026). $\pi 0.7$: A Steerable Robotic Foundation Model. *Public blog post / technical communication*, released 2026-04-16. [verified 2026-05-29 via Webfetch of pi.website]

21. Octo Model Team; Ghosh, D., Walke, H., Pertsch, K., Black, K., Mees, O., Dasari, S., Hejna, J., Kreiman, T., Xu, C., Luo, J., Tan, Y. L., Chen, L. Y., Sanketi, P., Vuong, Q., Xiao, T., Sadigh, D., Finn, C., & Levine, S. (2024). Octo: An Open-Source Generalist Robot Policy. arXiv:2405.12213 (submitted 2024-05-20, last revised v2 2024-05-26). Project page: https://octo-models.github.io [verified 2026-05-30 — arXiv ID + 19-author full list confirmed]

22. Hugging Face LeRobot Project (2024–2026). *LeRobot: State-of-the-art Machine Learning for Real-World Robotics*. Open-source software project. Includes the SmolVLA and VLA-JEPA model lines. Latest version: v0.5.1 (2026-04-07); 24.5k GitHub stars; Apache-2.0 license. Repository: https://github.com/huggingface/lerobot. Model hub: https://huggingface.co/lerobot [verified 2026-05-29]

23. NVIDIA GR00T Team; Fan, L. "Jim", Zhu, Y., et al. (2025). GR00T N1: An Open Foundation Model for Generalist Humanoid Robots. arXiv:2503.14734. (41 authors alphabetical; current model family in May 2026 is GR00T N1.7.) [verified 2026-05-29]

24. Liu, S., Wu, L., Li, B., Tan, H., Chen, H., Wang, Z., Xu, K., Su, H., & Zhu, J. (2024). RDT-1B: A Diffusion Foundation Model for Bimanual Manipulation. arXiv:2410.07864. (Tsinghua University; ICLR 2025 publication.) [verified 2026-05-29]

25. Kim, M. J., Pertsch, K., Karamcheti, S., Xiao, T., Balakrishna, A., Nair, S., Rafailov, R., Foster, E., Lam, G., Sanketi, P., Vuong, Q., Kollar, T., Burchfiel, B., Tedrake, R., Sadigh, D., Levine,

S., Liang, P., & Finn, C. (2024). OpenVLA: An Open-Source Vision-Language-Action Model. arXiv:2406.09246. (Stanford / UC Berkeley / Toyota Research Institute; 18 authors.) [verified 2026-05-29]

26. Figure AI (2025). Helix: A Vision-Language-Action Model for Generalist Humanoid Control. *Public technical announcement*, released 2025-02-20. https://www.figure.ai/news/helix [verified 2026-05-30 — exact title and release date confirmed; prior reference rendering "Generalist Foundation Model for Humanoid Manipulation" corrected to official title]

27. Aubakirova, D., Marafioti, A., Roy Gosthipaty, A., Capuano, F., Ben Allal, L., Cuenca, P., Shukor, M., Cadene, R., et al. (2025). SmolVLA: Efficient Vision-Language-Action Model trained on Lerobot Community Data. *Hugging Face Blog*, released 2025-06-03. SmolVLA-450M parameter model (SmolVLM2-500M + ~100M action expert) trained on 487 community datasets. https://huggingface.co/blog/smolvla Companion paper: https://huggingface.co/papers/2506.01844 [verified 2026-05-30 — blog post format, title, date, author list, parameter counts all confirmed; prior reference category "Technical Report" corrected to "Blog Post"]

27a. Chi, C., Xu, Z., Pan, C., Cousineau, E., Burchfiel, B., Feng, S., Tedrake, R., & Song, S. (2024). Universal Manipulation Interface: In-The-Wild Robot Teaching Without In-The-Wild Robots. arXiv:2402.10329. (Hand-held gripper demonstration-capture interface plus policy-learning pipeline; venue not stated on the arXiv abstract page, confirm RSS 2024 at editorial polish.) [verified 2026-06-01 via WebFetch of arxiv.org/abs/2402.10329 — title, 8-author list, arXiv ID, and 2024 year confirmed]

8.5 E. Robotics platforms and standards

28. Open Robotics. *ROS 2 Documentation*. (Continuously updated documentation set; current stable release Lyrical Luth, released May 2026. Recent LTS releases: Kilted Kaiju 2025, Jazzy Jalisco, Humble Hawksbill.) https://docs.ros.org/ [verified 2026-05-29]

29. Open Source Robotics Foundation (2014–present). *ROS 2 Design Documents*. GitHub repository (Jekyll site, ros2 organization). https://github.com/ros2/design [verified 2026-05-30 — 240 stars, 195 forks, hosted at design.ros2.org]

30. ISO 10218-1:2025. *Robotics — Safety requirements — Part 1: Industrial robots*. International Organization for Standardization. Published February 2025, replacing ISO 10218-1:2011. Incorporates content of former ISO/TS 15066 (collaborative-robot safety) and adds cybersecurity requirements. [verified 2026-05-29]

31. ISO 10218-2:2025. *Robotics — Safety requirements — Part 2: Industrial robot applications and robot cells*. International Organization for Standardization. Published February 2025, replacing ISO 10218-2:2011. Note: 2025 edition title differs from the 2011 edition's "Robot systems and integration". [verified 2026-05-29]

32. ISO 13482:2014. *Robots and robotic devices — Safety requirements for personal care robots*. Interna-

tional Organization for Standardization. Covers human-robot physical contact applications; limited to earthbound robots. [verified 2026-05-29]

33. Open Robotics. *URDF (Unified Robot Description Format)*. Reference implementation: `urdfdom` (URDF parser core data structures and XML parsers; version 3.0.0 October 2021; supports URDF 1.0/1.1/1.2 with backward compatibility). https://github.com/ros/urdfdom [verified 2026-05-30] (Note: the canonical specification page at wiki.ros.org/urdf is currently behind Anubis security gate; the urdfdom repository is the authoritative reference implementation.)

34. Google DeepMind. *MJCF (MuJoCo XML Format) Specification*. MuJoCo Documentation, stable version. 180+ XML elements with hierarchical relationships covering option/compiler/assets/kinematic-tree/sensors/actuators/contacts/visual. https://mujoco.readthedocs.io/en/stable/XMLreference.html [verified 2026-05-29]

8.6 F. Hardware embodiments cited in the worked example and discussion

35. Shaw, K., Agrawal, A., & Pathak, D. (2023). LEAP Hand: Low-Cost, Efficient, and Anthropomorphic Hand for Robot Learning. *Robotics: Science and Systems (RSS) 2023*. Carnegie Mellon University. Project page: https://v1.leaphand.com/ [verified 2026-05-30 — RSS 2023 venue, CMU affiliation, full 3-author list confirmed; **author surname correction: Agrawal (not Agarwal) per official project page**; leaphand.com 302-redirects to v1.leaphand.com]

36. Wonik Robotics Co., Ltd. *Allegro Hand* (current V5 product line: V5 Sense with 16 pressure sensing channels; V5 Plus with 16-DOF; V5 with 9-DOF; legacy V4 still supported). Official product page: https://www.allegrohand.com/ [verified 2026-05-30 — URL corrected from `wonik.com` to `allegrohand.com` per Wonik Robotics official site]

37. Shadow Robot Company. *Shadow Dexterous Hand (v4) Technical Specification*. 24 movements / 20 DOF tendon-driven; 20 motors; 100+ sensors at 1KHz; self-contained actuation in hand and forearm; current commercial v4 series released 2022. https://shadowrobot.com/dexterous-hand-series/ [verified 2026-05-29]

38. Bridgestone Soft Robotics Ventures (2026). *TETOTE: A Universal Robot Hand Series* (4 models: Slim, Standard, Strong-6, Strong-12). https://www.bridgestone.co.jp/products/softrobotics/tetote/ [verified 2026-05-29]

39. Tesla, Inc. (2024–2026). Optimus Hand and Arm Specifications across generations.

    - **Gen 2 reference patent**: Leddy, M. (assignee Tesla Inc.). "Underactuated hand with cable-driven fingers." WIPO Publication WO2024/073138 A1. Filed 2023-10-02; published 2024-04-04. https://patents.google.com/patent/WO2024073138A1 [verified 2026-05-30 — Google Patents]
    - **Gen 3 patent family**: Four international WIPO patents covering forearm, wrist, joint, and hand architecture; priority date 2024-10-10 (filed on the day of Tesla's "We, Robot" event); published 2026-04-16. Lead inventor on "Robotic Forearm Assembly": Kon-

stantinos Laskaris (Tesla's former chief motor designer, now leading the Optimus program). 25 linear actuators (23 hand + 2 wrist) arranged in concentric rings around a central rotary actuator; 22-DOF hand (4 DoF × 5 fingers + 2 wrist) + 3 DoF wrist/forearm = 25 DoF per arm; 50 actuators total per robot. [partial verify 2026-05-30 — patent family count, priority date, publication date, and lead inventor confirmed; specific WIPO publication numbers per individual patent not yet indexed in public search results at verification time]

- **Important caveat**: Per Musk disclosure (2026-04-20, Twitter/X), Tesla has "already abandoned" the Gen 3 rolling-contact design after sim-to-real failures, and the V3 hands at the summer 2026 reveal will use a newer, undisclosed design.
- Corporate announcements: Musk Gen 3 hand video disclosure 2026-02-14; specs announcement 2026-02-17.

40. Figure AI (2024). Figure unveils Figure 02, its second-generation humanoid, setting new standards in AI and robotics. *Official press release via PR Newswire*, 2024-08-06. Key specs: 16-DOF hands with human-equivalent strength, 2.25 kWh custom battery pack (50%+ energy improvement over Figure 01), 3x CPU/GPU capacity vs prior generation, 6 onboard RGB cameras. https://www.prnewswire.com/news-releases/figure-unveils-figure-02-its-second-generation-humanoid-setting-new-standards-in-ai-and-robotics-302214889.html [verified 2026-05-30]

40a. Figure AI (2025). *Introducing Figure 03*. Public technical announcement released 2025-10-09. https://www.figure.ai/news/introducing-figure-03 (Figure 02 referenced comparatively: "9% less mass and significantly less volume than Figure 02", "2x faster speeds with improved torque density".) [verified 2026-05-30]

41. Sanctuary AI (2024–2025). *Phoenix* (current public generation: **Generation 7**, launched April 2024 with improved task learning <24 hours, range of motion, weight, and bill-of-materials cost; subsequent Gen 8 data-capture variant per January 2025 announcement). Specs (Gen 7): 170 cm height, 70 kg weight, 20-DOF hands with haptic feedback, 25 kg payload, hydraulic hand actuation. https://www.sanctuary.ai/ [verified 2026-05-30 — official site lists Gen 7 as current public-product generation; Gen 8 referenced separately as data-capture release]

42. Apptronik (2025). *Apollo Humanoid Specifications*. Height 5'8" (~173cm), weight 160 lbs (~73kg), payload 55 lbs (~25kg), 4-hour battery with hot-swap. Marketed as "the first commercial humanoid robot." Target deployment: warehouses and manufacturing plants near-term; construction/oil & gas/electronics/retail/delivery/elder care future. https://apptronik.com/apollo [verified 2026-05-29]

43. Unitree Robotics. *G1 and H1 Humanoid Specifications*. G1: 1320 mm height, ~35 kg weight (with battery), 23 DOF, ~2 kg arm payload, ~2-hour battery, from $13.5K. https://www.unitree.com/g1 H1: ~180 cm height, ~47 kg, 18 DOF (4/arm, 5/leg), 3.3 m/s max speed, 189 N·m/kg peak torque density. H1-2 successor variant ~178 cm, ~70 kg, 27 DOF. https://www.unitree.com/h1 [verified 2026-05-30]

44. Seed Robotics. *RH8D Adult Robot Hand Specifications*. 19 DOF; 8 actuators; 1 kg 3D payload (2.5 kg vertical); 620 g; 9-24V; optional tactile pressure sensors (1 mN resolution). https://www.seedrobotics.com/rh8d-adult-robot-hand [verified 2026-05-29] (note: marketed as both "Inspire RH8D" and "Seed Robotics RH8D" in references; correct vendor is Seed Robotics)

8.7 G. Coordination layers in comparative analysis (§ 3, § 5, § 8)

45. Furber, S. (2000). *ARM System-on-Chip Architecture* (2nd ed.). Addison-Wesley. ISBN-13: 9780201675191. [verified 2026-05-30 — 2nd edition, 2000, Addison-Wesley publication confirmed]

46. ARM Limited. *Arm Architecture Reference Manual for A-profile architecture*. Current architecture revision: Armv9.7-A (2025), with copyright spanning 2020 and 2022–2025. https://developer.arm.com/documentation/ddi0487/latest [verified 2026-05-29]

47. USB Implementers Forum. *USB 4 Specification* (Version 2.0). Signaling rates 20/40/80 Gbit/s; USB-C-only connector. https://www.usb.org [verified 2026-05-29]

48. NVIDIA Corporation. *CUDA C++ Programming Guide*, Release 13.3 (2026-05-21). Features: CUDA Tile C++, CompileIQ AI-powered auto-tuning, CUDA Python 1.0. https://docs.nvidia.com/cuda/cuda-c-programming-guide/ [verified 2026-05-29]

49. The Open Group / IEEE (2024). *POSIX (IEEE Std 1003.1-2024) — Portable Operating System Interface, Base Specifications Issue 8*. Published 2024-06-14, aligned with C17 language standard. https://standards.ieee.org/ieee/1003.1/7700/ [verified 2026-05-29]

8.8 H. Structural precedents for white paper form

50. Nakamoto, S. (2008). Bitcoin: A Peer-to-Peer Electronic Cash System. (Self-published 9-page white paper, distributed via cryptography mailing list, 2008-10-31.) [Referenced for rhetorical structure, not technical content; verified 2026-05-29]

51. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems 30 (NIPS / NeurIPS 2017)*. arXiv:1706.03762. [Referenced for academic abstract register; verified 2026-05-29]

8.9 I. Industry analyst and market estimates (§ 1) — DELETED IN v1.3

Entries 52, 53, and 54 (placeholder slots for PitchBook / CB Insights VC financing aggregates, industry-analyst "suppressed market value" estimates, and 2026 robotics-AI engineering compensation surveys) were **deleted in v1.3 (2026-05-30)**. The quantitative claims those placeholders would have supported are resolved in the body via **Path A (soften)** — see `appendix_A_data_sources_en.md § A.4--A.6` for the full softening record. No body citation depends on these entries; deletion is safe.

9 Appendices

10 Appendix A. TactileManifold: Formal Specification

The TactileManifold is the RFL Translation Layer's canonical representation of tactile state. Its design satisfies a constraint that is unmet by any existing tactile-sensor format: it must express the observations of a binary four-channel contact sensor, a three-axis force-torque sensor, a high-resolution GelSight-class visuotactile sensor, and as-yet-undesigned tactile modalities in a single uniform structure that admits capability negotiation. This appendix gives the formal definition, the resolution hierarchy that enables negotiation, and the realizations for the principal sensor classes in 2026 commercial use.

10.1 A.1 Motivation and Design Constraints

The body section establishes the structural requirement: a VLA expressing a high-level skill (e.g., `grasp_pinch`) must be able to specify a tactile target without committing to a particular sensor format. An embodiment receiving the target must be able to report its observations against it. Different embodiments with different sensors must produce comparable observations. The abstraction therefore satisfies three constraints simultaneously:

1. **Format independence**: a sensor's native output format does not appear in the manifold definition. Sensor-specific encoding is performed by the embodiment's Driver Interface during translation.
2. **Resolution heterogeneity**: a low-resolution sensor and a high-resolution sensor must express observations in the same abstraction, with the resolution difference captured as a parameter rather than as a different structure.
3. **Graceful degradation**: if a VLA requests a feature the embodiment cannot supply, the negotiation must fail cleanly with a structured error, not silently produce a degraded observation that the VLA misinterprets.

The third constraint is the load-bearing one. Many existing tactile-fusion approaches produce a single output that hides the underlying resolution differences, with the consequence that a VLA trained on high-resolution data degrades unpredictably when deployed against a low-resolution sensor. The TactileManifold abstraction is designed to make the resolution explicit at every layer.

10.2 A.2 Formal Definition

A TactileManifold is a tuple

$$\mathcal{M} = (S, \mathcal{F}, R, t)$$

where:

- $S \subseteq \mathbb{R}^3$ is a *spatial domain*, a (possibly discrete) subset of the embodiment's end-effector surface, equipped with a topology induced by the surface metric. For dense sensors, S is a two-dimensional manifold patch; for discrete sensors, S is a finite set of points; both are admitted by the definition.
- $\mathcal{F} : S \to \mathbb{V}$ is a *feature field*, assigning to each point in S a vector of contact features drawn from a *feature vocabulary* $\mathbb{V}$. The vocabulary is defined below and is the structure that admits cross-modality comparison.
- $R = (r_s, r_t, r_f)$ is a *resolution triple*, where r_s is the spatial density (points per unit area, or "discrete" for sparse sensors), r_t is the temporal sample rate (Hz), and $r_f : \mathbb{V} \to \mathbb{N}$ is a per-feature precision function (bits of effective dynamic range per feature channel).
- $t \in \mathbb{R}_{\geq 0}$ is the timestamp of the manifold instance.

The feature vocabulary $\mathbb{V}$ is defined as a fixed, extensible set of named feature channels:

$$\mathbb{V} = \{\text{contact}, \text{normal\_force}, \text{shear}_x, \text{shear}_y, \text{slip}, \text{vibration}, \text{temperature}, \text{compliance}, \ldots\}$$

Each named channel has a fixed semantic interpretation specified in the RFL spec document. The set is extensible through the extension registry (§ 6.1), so future sensor modalities can register new channels without breaking the core specification.

A *partial manifold* is an instance in which $\mathcal{F}$ is defined only over a subset of the vocabulary. The spec language requires the embodiment's capability manifest to declare which channels it supplies, so the partial nature is contractual and verifiable rather than silent.

10.3 A.3 The Resolution Hierarchy

The resolution triple R admits a partial order $\succeq$:

$$R_1 \succeq R_2 \iff r_{s,1} \geq r_{s,2} \ \wedge \ r_{t,1} \geq r_{t,2} \ \wedge \ r_{f,1}(c) \geq r_{f,2}(c) \ \forall c \in \mathbb{V}$$

This partial order is the structure that supports capability negotiation. When a VLA emits a Skill Layer invocation with a tactile target specified at resolution R_{req}, the Translation Layer queries the embodiment's capability manifest for the native resolution R_{native}. If $R_{\text{native}} \succeq R_{\text{req}}$, the invocation proceeds; the embodiment supplies a manifold instance at R_{native} (or, optionally, downsamples to R_{req} to reduce bandwidth). If $R_{\text{native}} \not\succeq R_{\text{req}}$, the negotiation fails with a structured error identifying the dimension(s) on which the requested resolution exceeds the embodiment's capability.

The asymmetric nature of the partial order is intentional. A VLA that requests low resolution and receives high resolution can use the additional information or discard it without error. A VLA that requests high resolution and receives low resolution cannot infer the missing information; the

failure must be visible. The Translation Layer enforces this by treating the resolution declaration as a contract rather than a hint.

10.4 A.4 Realizations from Specific Sensor Classes

Five sensor classes in 2026 commercial use are realized as follows. The mapping shown for each is the canonical one defined in the RFL Reference Implementation; embodiment vendors may register alternative mappings through the extension registry if their sensor admits more refined representation.

Binary contact sensors (four-channel discrete). Used in many low-cost grippers. Mapping: S is a discrete four-point set corresponding to fingertip locations; $\mathcal{F}(s) = \{\text{contact: } \{0, 1\}\}$ for each $s \in S$; $R = (\text{discrete}, 100\text{ Hz}, r_f(\text{contact}) = 1)$. Only the contact channel is populated; the remaining vocabulary is undefined.

Three-axis force-torque sensors at the wrist. Used in most collaborative arms. Mapping: S is a single-point set at the force-sensor location; $\mathcal{F}(s) = \{\text{normal\_force}, \text{shear}_x, \text{shear}_y\}$ as three scalars; $R = (\text{discrete}, 1000\text{ Hz}, r_f(\cdot) = 16)$. The three force channels are populated; spatial channels are absent because the sensor is point-localized.

Distributed force-sensor arrays (XELA-class). Used in high-end multi-finger hands. Mapping: S is a discrete set of sensor locations (typically 8–24 per fingertip); $\mathcal{F}(s) = \{\text{normal\_force}, \text{shear}_x, \text{shear}_y\}$ at each location; $R = (\text{discrete}_n, 100\text{ Hz}, r_f(\cdot) = 14)$ where n is the per-fingertip array size.

Visuotactile sensors (GelSight, DIGIT classes). Used in research-grade and emerging industrial deployments. Mapping: S is a dense two-dimensional patch (e.g., 320×240 pixels mapped to the elastomer-gel surface); $\mathcal{F}(s) = \{\text{normal\_force}, \text{shear}_x, \text{shear}_y, \text{slip}\}$ derived from the gel-deformation image; $R = (\text{dense}, 30\text{ Hz}, r_f(\cdot) = 8)$.

Pneumatic pressure sensors (as in McKibben and bellows-style pneumatic hands). Used in pneumatic-hand embodiments. Mapping: S is a discrete set of finger-chamber locations (e.g., six chambers in a six-finger pneumatic hand); $\mathcal{F}(s) = \{\text{normal\_force derived from chamber pressure}\}$; $R = (\text{discrete}, 50\text{ Hz}, r_f(\text{normal\_force}) = 12)$. The pressure-to-force conversion is calibrated through the embodiment descriptor's calibration metadata.

This list is not exhaustive; the design intent is that any tactile sensor whose output can be characterized by a spatial distribution of feature values can be expressed in TactileManifold form. The capability negotiation mechanism then handles the heterogeneity at runtime.

10.5 A.5 Capability Negotiation Semantics

For a Skill Layer primitive whose tactile target is $\mathcal{M}_{\text{req}} = (S_{\text{req}}, \mathcal{F}_{\text{req}}, R_{\text{req}}, \cdot)$, the negotiation against an embodiment with native capability $\mathcal{M}_{\text{native}} = (S_{\text{native}}, \mathcal{F}_{\text{native}}, R_{\text{native}}, \cdot)$ proceeds in three steps:

1. **Feature coverage**: every channel in $\mathcal{F}_{\text{req}}$ must be present in $\mathcal{F}_{\text{native}}$. If not, negotiation fails with `UnsupportedFeatureError(channels=...)`.
2. **Spatial coverage**: S_{req} must be approximable by points in S_{native} within a declared spatial tolerance. If not, negotiation fails with `SpatialCoverageError(uncovered_region=...)`.
3. **Resolution adequacy**: $R_{\text{native}} \succeq R_{\text{req}}$ as defined in A.3. If not, negotiation fails with `InsufficientResolutionError(dimensions=...)`.

The Driver Interface returns the structured error rather than silently substituting a fallback. The VLA's response to negotiation failure is its own concern; the Translation Layer's responsibility is to make the failure visible and structured.

The negotiation produces, on success, an *adapted manifold* $\mathcal{M}_{\text{adapted}}$ which is the embodiment-native manifold downsampled to R_{req} (if downsampling is requested) or supplied at R_{native} (the default). The VLA receives observations against $\mathcal{M}_{\text{adapted}}$, with a metadata field indicating which resolution was used.

10.6 A.6 Open Questions

Two questions are left to future spec revisions.

First, the formal treatment of *deformable-object tactile interaction* — where the contact surface itself deforms during interaction and the spatial domain S should be time-varying — is sketched but not specified. The current spec accepts S as time-invariant during a single primitive invocation; future revisions will need to address adaptive spatial domains for cloth, soft-object, and slip-rich manipulations.

Second, the relationship between TactileManifold and the *proprioceptive* state captured separately in the canonical action representation (§ 4.2) admits some overlap (force at the wrist appears in both). The current spec treats them as separate channels; a future revision may unify them under a richer end-effector-state abstraction.

These are flagged for v2.0 of the specification, consistent with the extension-registry policy (§ 6.1) by which the v0.1 / v1.0 core stays small and well-defined while non-trivial new structure migrates through extensions before promotion.

---

11 Appendix B. Supermodularity of Bilateral Translation

11.1 B.1 Setup

Let $T(a; \theta, \phi)$ denote the Translation Layer's mapping from a VLA action a to an embodiment-specific command, parameterized by θ (the layer's information about the VLA's semantics) and ϕ (its information about the embodiment). Let $V(\theta, \phi) = \mathbb{E}_{a,(\xi,\tau)}[U(T(a; \theta, \phi))]$ be the expected utility of the translation under a concave bounded utility U over a representative distribution over actions and task contexts.

11.2 B.2 Conditional Supermodularity

Claim. Assume standard regularity (boundedness of U, joint continuity and differentiability of T) and the substantive assumption

> **A** (constructive complementarity): $\partial^2 T / \partial\theta\, \partial\phi \geq 0$ in the relevant region of the domain — i.e., improvements in VLA-side and embodiment-side information combine constructively in the translation rather than substituting.

Then the value function V is supermodular in (θ, ϕ) — for $\theta_L < \theta_H$ and $\phi_L < \phi_H$,

$$V(\theta_H, \phi_H) - V(\theta_H, \phi_L) \geq V(\theta_L, \phi_H) - V(\theta_L, \phi_L)$$

— **unconditionally when U is risk-neutral** (locally linear), and under a concave (risk-averse) U whenever the complementarity term dominates the concavity term identified next.

The argument follows Topkis (1998), Theorem 2.6.1. The cross-partial of V decomposes into a *complementarity* term — non-negative, and the channel through which A enters — and a *concavity* term $U''(T)\, T_\theta T_\phi$, which is **non-positive** under a concave U because the two single-side partials share sign (so their product is non-negative while $U'' \leq 0$). Supermodularity therefore requires the complementarity term to dominate the concavity term: this holds trivially for risk-neutral U (where the concavity term vanishes) and, under risk aversion, in the regime A identifies — where joint information removes first-order variance that neither side removes alone. Integrating the resulting non-negative cross-partial over the rectangle $[\theta_L, \theta_H] \times [\phi_L, \phi_H]$ yields the inequality above.

11.3 B.3 What This Establishes

A is the load-bearing claim and is not proved here. The informal motivation is that a translator with joint information can exploit the joint distribution of (VLA action, embodiment response) rather than marginalize over a missing dimension — but this is an architectural argument about what optimal translation would look like, not a derivation. A is expected to hold in the generic case and to weaken in regions of (θ, ϕ) space where VLA-side and embodiment-side information happen to be informationally redundant; characterizing those regions is empirical work the appendix does not undertake.

The conditional supermodularity claim, then, is best read as a *design rationale* for why the Translation Layer is structured bilaterally (VLA model + embodiment model fed into one translator) rather than as two independent unilateral mappings: *if* A holds in deployment, the bilateral structure recovers compounded value that the unilateral structure cannot, and *if* A does not hold, the bilateral structure costs nothing relative to the unilateral structure (the translator can simply ignore one side). The argument is asymmetric in favor of the bilateral choice without depending on the supermodular form actually obtaining.

The spec's commitments in § 4 do not invoke this appendix as a premise. The bilateral Translation Layer is independently justifiable on the joint-information argument of the previous paragraph;

the supermodular formalization is included because the underlying claim is precise enough to be worth stating precisely and contesting precisely, not because it is part of the specification's testable surface.

End of RFL Specification v0.1